

AI Meeting Intelligence Platform

*A Self-Hosted, Privacy-First System for Meeting Transcription,
AI Summarisation, Speaker Analytics, and Q&A*

Submitted by

Shubhada Palwe

Roll No: 241770041

Under the Guidance of

[Ritambra Korpai]

Department of Computer Science

Savitribai phule university pune

MSc Computer Science — Final Year Project Report

Academic Year: 2025–2026

ABSTRACT

This project builds an AI-powered meeting platform that runs entirely on a local network, without sending any data to the internet. The system lets people hold video meetings, automatically record audio, generate a written transcript of who said what, and produce an AI-written summary of the meeting — all without using any paid cloud services.

The platform has a four-step transcription system: it first tries Sarvam AI (good for Indian languages), then AssemblyAI, then WhisperX, and finally falls back to faster-whisper which runs on any laptop CPU. This means a transcript is always produced, even without an internet connection.

After transcription, the system can generate a meeting summary, answer questions about the meeting, show speaking-time statistics for each participant, create a PDF of the meeting minutes, and send the PDF to participants by email.

The entire system is packaged using Docker, so it can be started with a single command and accessed from any device on the same Wi-Fi network. The project was tested with two participants and all features worked correctly.

TABLE OF CONTENTS

Acknowledgement.....	
Abstract.....	
List of Figures.....	
Chapter 1 Introduction and Project Overview.....	
Chapter 2 Problem Statement.....	
Chapter 3 Existing Systems.....	
Chapter 4 Proposed System.....	
Chapter 5 Objectives.....	
Chapter 6 Scope of the Project.....	
6.1 Features Included.....	
6.2 Features Not Included.....	
Chapter 7 Technologies Used.....	
Chapter 8 System Requirements.....	
8.1 Software Requirements.....	
8.2 Hardware Requirements.....	
Chapter 9 System Architecture.....	
9.1 Overall System Architecture.....	
9.2 Meeting Lifecycle State Machine.....	
Chapter 10 Project Workflow.....	
10.1 End-to-End Meeting Flow.....	
10.2 Audio Processing Flow.....	
Chapter 11 Module-wise Description.....	
11.1 Startup and Deployment.....	
11.2 Landing Page.....	
11.3 Create Meeting.....	
11.4 Invite Sharing.....	
11.5 Meeting Room.....	
11.6 Active Speaker and Screen Sharing.....	
11.7 Audio Recording.....	
11.8 Transcription.....	
11.9 AI Summary.....	
11.10 Email Digest.....	
11.11 Ask AI (Q&A).....	
Chapter 12 Database Design.....	
Chapter 13 API Design.....	
13.1 API Endpoints.....	
13.2 Frontend Components.....	
13.3 WebRTC Flow.....	

Chapter 14 Implementation Details.....

14.1 Folder Structure.....

14.2 Key Coding Patterns.....

14.3 Transcription Pipeline Code.....

Chapter 15 Earlier Session Screenshots.....

Chapter 16 Testing.....

Chapter 17 Results.....

Chapter 18 Discussion.....

Chapter 19 Conclusion.....

Chapter 20 Future Work.....

References.....

Appendix A: Environment Variables.....

Appendix B: API Quick Reference.....

CHAPTER 1: INTRODUCTION AND PROJECT OVERVIEW

Meetings are an important part of every organisation, college, or team. But one big problem is that after a meeting ends, people often forget what was discussed, who said what, or what decisions were made. Writing notes by hand during a meeting is difficult and people miss important points. Asking someone to sit and type the full conversation is time-consuming and expensive.

This project solves that problem by building an AI-powered meeting platform that automatically records meetings, converts the speech to text, and uses Artificial Intelligence to create a short summary — all without using any paid internet service.

The most unique part of this project is that everything runs on a local computer or local network. No meeting audio or transcript is ever uploaded to any third-party server. This makes the system completely private and safe to use even for sensitive discussions.

1.1 What Does the System Do?

When a meeting starts, the system records the audio. After the meeting ends, it converts that audio to a written transcript showing who spoke and what they said. It then creates a neat summary of the meeting, allows users to ask questions about the meeting, and can send a PDF of the minutes to all participants by email.

1.2 Why Is This Useful?

Students, teachers, company employees, and medical professionals all have meetings. All of them face the same problem of losing track of what was discussed. This platform gives them a simple, free, private way to record and remember every meeting automatically.

CHAPTER 2: PROBLEM STATEMENT

Current meeting tools like Zoom, Google Meet, or Microsoft Teams require a paid subscription to get AI features like transcripts and summaries. They also send all your audio and data to servers owned by American companies. For organisations that deal with private information — like hospitals, schools, or government offices — this is a serious privacy risk.

Even when these tools are used, the transcripts produced are not always accurate for Indian speakers of English, or for meetings where people switch between Hindi and English. The AI summaries are generic and do not always capture the key decisions made in a meeting.

There is no free, open-source, privacy-first solution that a student or small organisation can set up on their own computer without technical expertise. This project aims to fill that gap.

CHAPTER 3: EXISTING SYSTEMS AND THEIR LIMITATIONS

3.1 Zoom AI Companion

Zoom offers an AI meeting assistant that creates summaries and transcripts. However, it requires a paid Pro plan, sends all audio to Zoom servers in the USA, and does not work well with Indian English accents or code-switching between languages.

3.2 Otter.ai

Otter.ai is a popular transcription tool. It works well for American English but struggles with Indian accents. It also requires an internet connection and stores recordings on its servers, which is a privacy concern.

3.3 Microsoft Teams Premium

Microsoft Teams Premium includes intelligent meeting recap. This requires an expensive Microsoft 365 Premium subscription and is tightly tied to the Microsoft ecosystem. It does not work for free or self-hosted environments.

3.4 Google Meet with Gemini

Google Meet with Gemini AI provides transcripts and summaries. Like the others, it requires a paid Workspace subscription and sends all data to Google servers.

3.5 How This Project Is Different

This project is completely free and open-source. It runs entirely on a local computer — no data leaves the user's network. It supports multiple transcription engines and can work even without an internet connection. It is the only solution in this comparison that gives full privacy control to the user.

CHAPTER 4: PROPOSED SYSTEM

The proposed system is a web-based video meeting platform that any person on the same Wi-Fi network can join by opening a link in their browser. There is nothing to install on the joining device — just open the link and the meeting starts.

The host creates the meeting from a simple dashboard. They get a shareable link that they can send via WhatsApp or email. Once all participants have joined and the meeting is over, the host clicks “Stop Recording”. The system then processes the audio in the background and presents a full transcript, speaker analytics, and an AI summary on the same page.

The system is packaged using Docker, a technology that bundles all software together so the host can start everything with a single command. This means even a non-technical user can set it up by following simple instructions.

CHAPTER 5: OBJECTIVES

The main objectives of this project are:

- To build a self-hosted video meeting system that works on a local network without any internet dependency.
- To automatically record meeting audio when enabled by the host.
- To transcribe the recorded audio into text using a four-step fallback pipeline that supports multiple AI engines.
- To identify which speaker said which part of the conversation (speaker diarisation).
- To generate a readable AI-written summary of the meeting after it ends.
- To allow participants to ask questions about the meeting and get AI answers.
- To generate a PDF of the meeting minutes that can be shared by email.
- To ensure that no meeting data is ever sent to any external server.

CHAPTER 6: SCOPE OF THE PROJECT

6.1 Features Included

- Browser-based video meetings using WebRTC (no app install needed for participants).
- Automatic audio recording when enabled by the host.
- Four-engine transcription fallback: Sarvam AI → AssemblyAI → WhisperX → faster-whisper.
- Speaker diarisation (who said what).
- AI-generated meeting summary using a language model.
- Ask AI — a Q&A panel where users can type questions about the meeting.
- PDF generation of full meeting minutes with transcript and summary.
- Email delivery of the PDF to specified recipients.
- Speaker analytics (how much time each person spoke).
- Screen sharing support.
- Docker-based deployment with a single-command startup.

CHAPTER 7: TECHNOLOGIES USED

7.1 Backend

FastAPI (Python) is used as the main web server. It handles API requests from the frontend, controls the recording process, and manages the transcription pipeline. FastAPI was chosen because it is fast, simple to write, and works well with async (non-blocking) operations which are needed for long audio processing tasks.

7.2 Video and Audio — LiveKit

LiveKit is an open-source WebRTC server. It handles all video and audio streams between participants. It also provides server-side recording, which allows the system to record all audio directly on the server without the user needing to install anything.

7.3 Frontend — React

The user interface is built with React and TypeScript. React allows building interactive UIs with reusable components. The LiveKit React SDK is used to connect the browser to the LiveKit server and display video/audio streams.

7.4 Transcription Engines

Four transcription engines are used in order of preference. Sarvam AI handles Indian languages well. AssemblyAI provides good accuracy for English with speaker diarisation. WhisperX is a faster version of OpenAI's Whisper model that runs locally. faster-whisper is the final fallback and runs on CPU, so it works without a GPU.

7.5 AI Summary and Q&A

The AI summary and Q&A features use a large language model (LLM). The system can be configured to use either a local Ollama model or a cloud API like OpenAI. By default it uses a local model to maintain privacy.

7.6 Database — SQLite

SQLite is used as the database. It is a file-based database that requires no separate server and is perfect for this kind of self-hosted application. It stores meeting records, participants, transcripts, and summaries.

7.7 Email — SMTP

Email is sent using Python's built-in SMTP library. It connects to a Gmail or other SMTP server using the credentials provided in the environment configuration.

7.8 PDF Generation — ReportLab

PDF meeting minutes are generated using the ReportLab library. It creates a well-formatted PDF with the meeting summary, full transcript, and speaker statistics.

7.9 Deployment — Docker and Docker Compose

The entire system is packaged using Docker. Docker Compose allows all services (FastAPI backend, LiveKit server, React frontend) to start with a single command: `docker-compose up`.

CHAPTER 8: SYSTEM REQUIREMENTS

8.1 Software Requirements

- Docker Desktop (Windows/Mac) or Docker Engine (Linux)
- Docker Compose version 2.0 or higher
- Node.js 18 or higher (for frontend development only)
- Python 3.10 or higher (for backend development only)
- A web browser — Chrome 90+, Firefox 88+, or Safari 14+
- Git (to clone the project repository)

8.2 Hardware Requirements

- Processor: Intel Core i5 or equivalent (i7 recommended for real-time processing)
- RAM: Minimum 8 GB (16 GB recommended when running WhisperX locally)
- Storage: At least 10 GB free disk space for Docker images and recorded audio
- Network: All devices must be on the same Wi-Fi or LAN network
- Microphone: Required on all participant devices
- Camera: Optional (audio-only meetings are supported)

CHAPTER 9: SYSTEM ARCHITECTURE

9.1 Overall System Architecture

The system has three main layers: the Frontend (React app in the browser), the Backend (FastAPI server), and the Infrastructure (LiveKit server + SQLite database). All three run together inside Docker containers and communicate with each other on the local network.

When a user opens the meeting page in their browser, the React frontend connects to the LiveKit server directly for video/audio using WebRTC. API calls for creating meetings, starting recordings, and fetching transcripts go to the FastAPI backend. The diagram below shows how all the parts connect to each other.

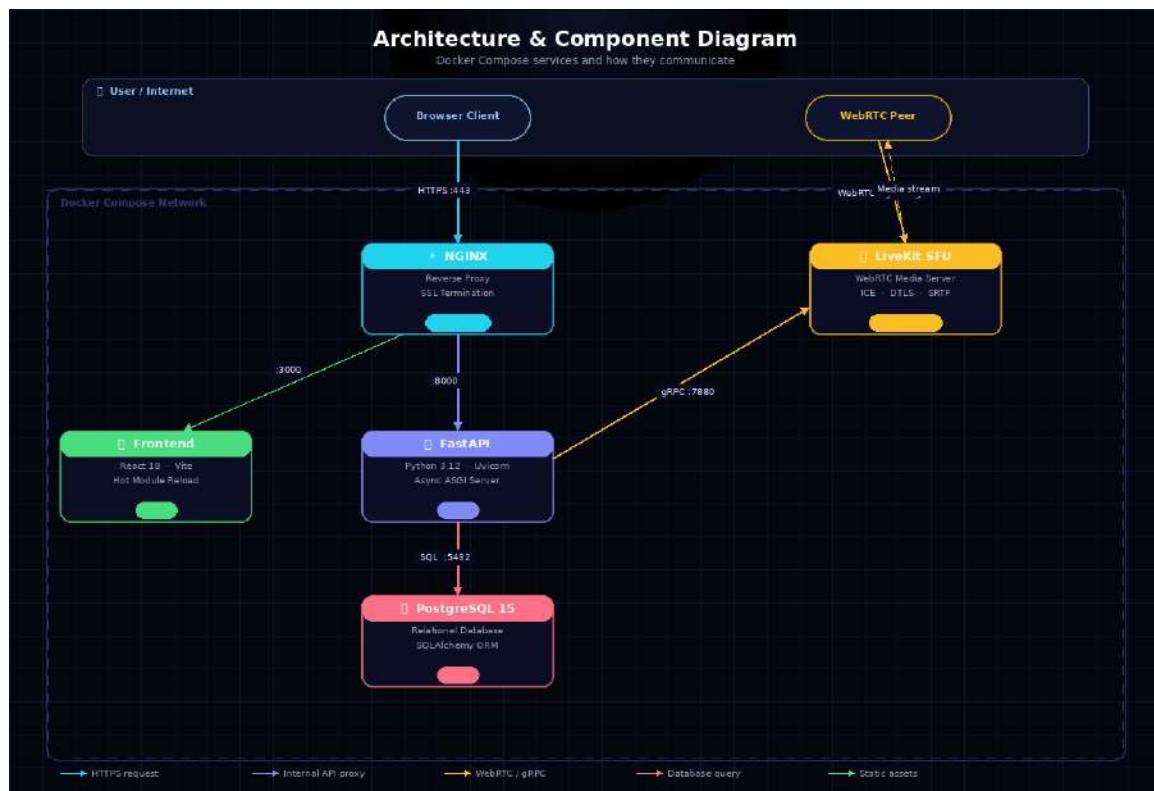


Figure 9.1: Overall System Architecture Diagram

9.2 Meeting Lifecycle State Machine

Every meeting in the system goes through a series of states. It starts as "Scheduled", moves to "Active" when participants join, then "Recording" when the host starts the recorder, then "Processing" when audio is being transcribed, and finally "Completed" when everything is done. The state machine below shows all possible states and the transitions between them.

Description

This diagram shows all the components of the system: the browser (React), the FastAPI backend, the LiveKit WebRTC server, the SQLite database, the transcription pipeline, and the email/PDF services. Arrows show how data flows between each component.

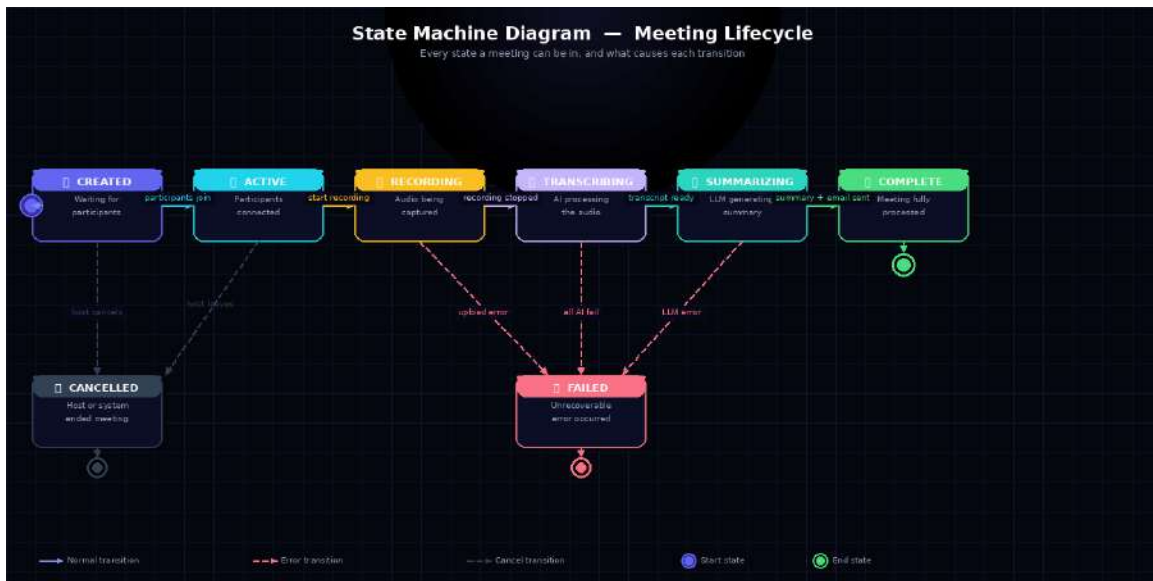


Figure 9.2: Meeting Lifecycle State Machine

Every meeting in the system goes through a series of states. It starts as "Scheduled", moves to "Active" when participants join, then "Recording" when the host starts the recorder, then "Processing" when audio is being transcribed, and finally "Completed" when everything is done. The state machine below shows all possible states and the transitions between them.

Description

This diagram shows the different states a meeting can be in (Scheduled, Active, Recording, Processing, Completed, Failed) and what events cause the system to move from one state to another.

Purpose

To show the complete lifecycle of a meeting from creation to completion.

CHAPTER 10: PROJECT WORKFLOW

10.1 Audio Processing Flow

When a meeting recording is stopped, the audio file is sent through a four-step transcription pipeline. Each step tries a different transcription engine. If one engine fails or returns empty results, the next one is tried automatically.

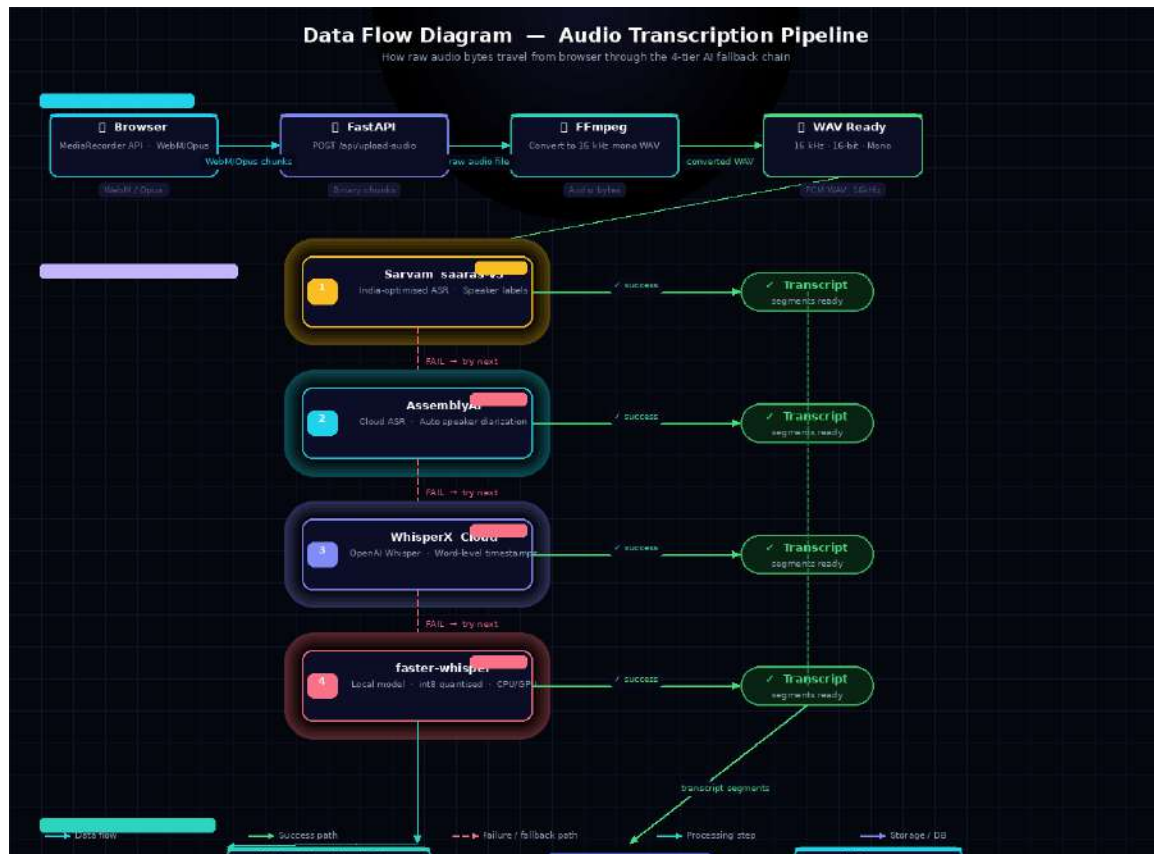


Figure 10.2: Audio Processing Data Flow Diagram

Description

A data flow diagram showing the four transcription stages: Stage 1 (Sarvam AI — best for Indian languages), Stage 2 (AssemblyAI — good for English with speaker labels), Stage 3 (WhisperX — runs locally, needs GPU), Stage 4 (faster-whisper — CPU-only fallback). Shows inputs, outputs, and error paths.

Purpose

To understand how the transcription pipeline works and how the system ensures a transcript is always produced even if one engine fails.

10.3 Meeting Join Workflow

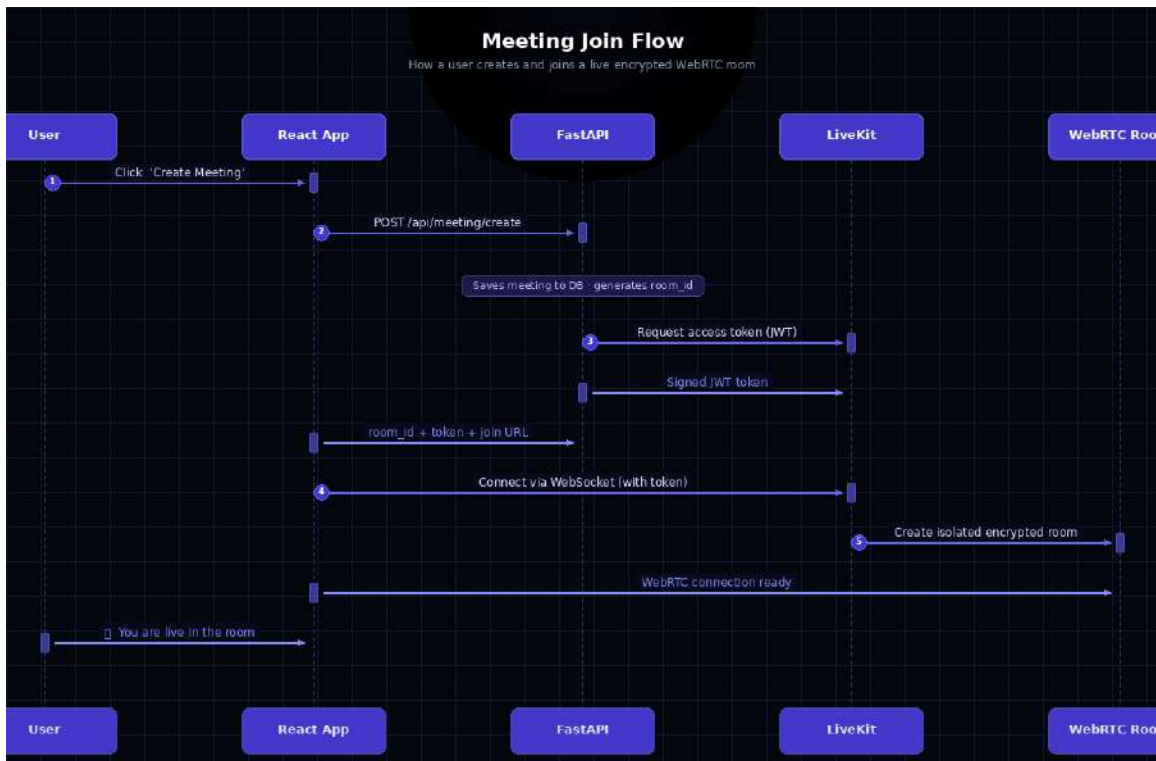


Figure 10.3: Meeting Join Workflow Diagram

Description

A flowchart showing the steps a participant follows to join a meeting: receive invite link → open in browser → enter name → click Join → connect to LiveKit WebRTC → appear in meeting room.

Purpose

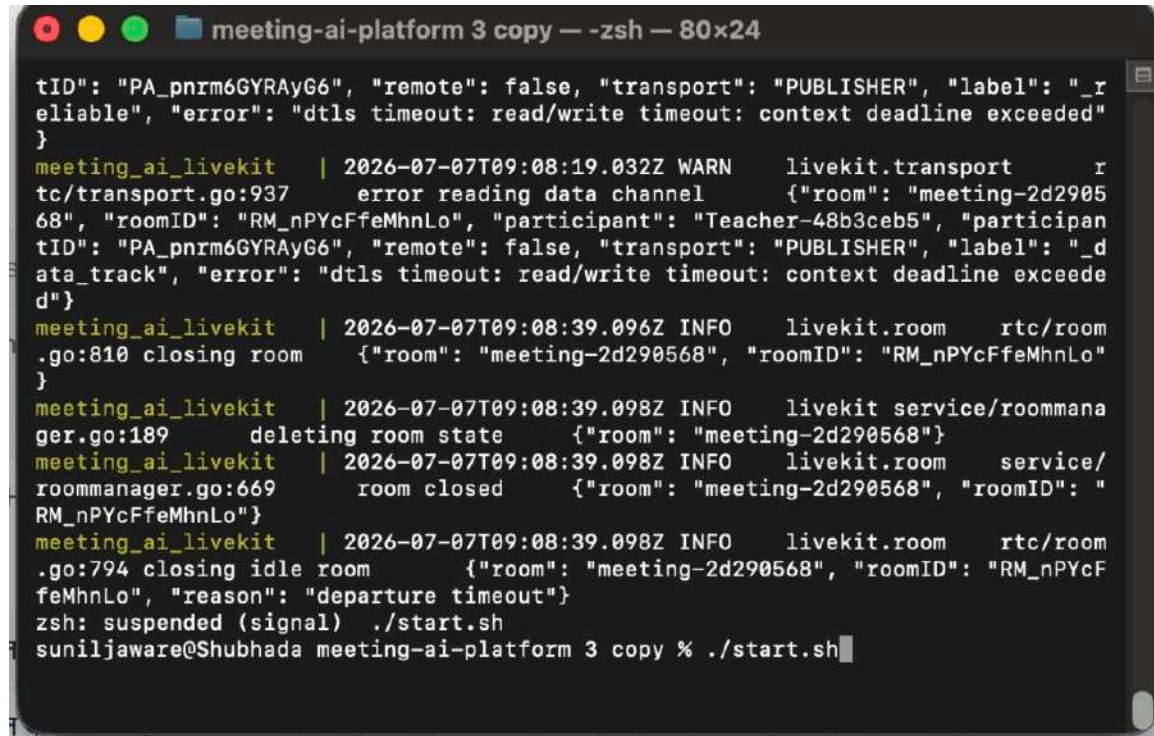
To show how simple the joining process is for participants — no app download, no account registration.

CHAPTER 11: MODULE-WISE DESCRIPTION

This chapter describes each feature (module) of the platform in detail. For each module, a screenshot from the actual running system is provided to show exactly what it looks like.

11.1 Startup and Deployment

The system starts by running the command `docker-compose up --build` from the project folder. Docker downloads all required software, builds the containers, and starts all services together. The terminal output shows a progress log of each service starting up.

A terminal window titled "meeting-ai-platform 3 copy - zsh - 80x24" displays the output of a Docker Compose command. The logs show the initialization of a LiveKit room. It starts with a warning from the transport module, followed by information from the room manager about creating and then closing a room. The process concludes with a "departure timeout" message and the user prompt.

```
tID": "PA_pnrm6GYRAyG6", "remote": false, "transport": "PUBLISHER", "label": "_r
eliable", "error": "dtls timeout: read/write timeout: context deadline exceeded"
}
meeting_ai_livekit | 2026-07-07T09:08:19.032Z WARN    livekit.transport    r
tc/transport.go:937    error reading data channel    {"room": "meeting-2d2905
68", "roomID": "RM_nPYcFfeMhnLo", "participant": "Teacher-48b3ceb5", "participan
tID": "PA_pnrm6GYRAyG6", "remote": false, "transport": "PUBLISHER", "label": "_d
ata_track", "error": "dtls timeout: read/write timeout: context deadline exceede
d"}
meeting_ai_livekit | 2026-07-07T09:08:39.096Z INFO    livekit.room    rtc/room
.go:810 closing room    {"room": "meeting-2d290568", "roomID": "RM_nPYcFfeMhnLo"
}
meeting_ai_livekit | 2026-07-07T09:08:39.098Z INFO    livekit service/roommana
ger.go:189    deleting room state    {"room": "meeting-2d290568"}
meeting_ai_livekit | 2026-07-07T09:08:39.098Z INFO    livekit.room    service/
roommanager.go:669    room closed    {"room": "meeting-2d290568", "roomID": "
RM_nPYcFfeMhnLo"}
meeting_ai_livekit | 2026-07-07T09:08:39.098Z INFO    livekit.room    rtc/room
.go:794 closing idle room    {"room": "meeting-2d290568", "roomID": "RM_nPYcF
feMhnLo", "reason": "departure timeout"}
zsh: suspended (signal) ./start.sh
suniljaware@Shubhada meeting-ai-platform 3 copy % ./start.sh
```

Figure 11.1: Docker Startup Log Screenshot

Description

Terminal screenshot showing Docker Compose starting all services. Lines show each container (livekit, backend, frontend) being built and started.

Purpose

To show that the system can be started with a single command.

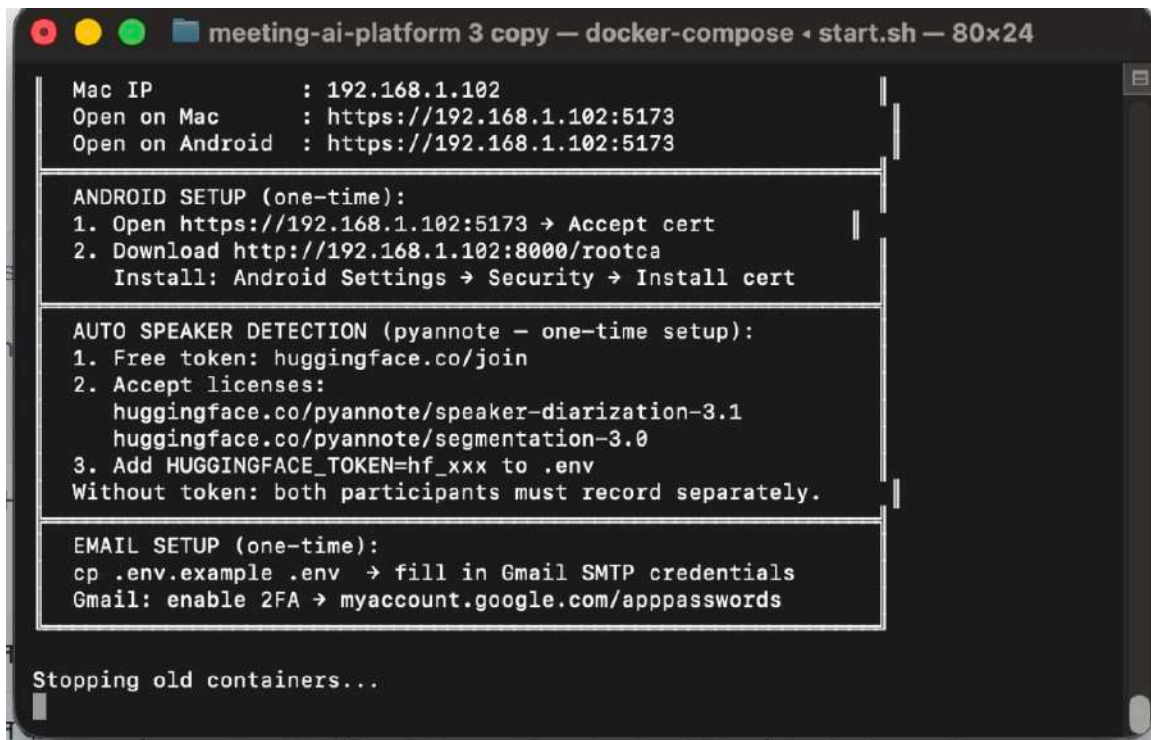


Figure 11.2: Startup Summary Screenshot

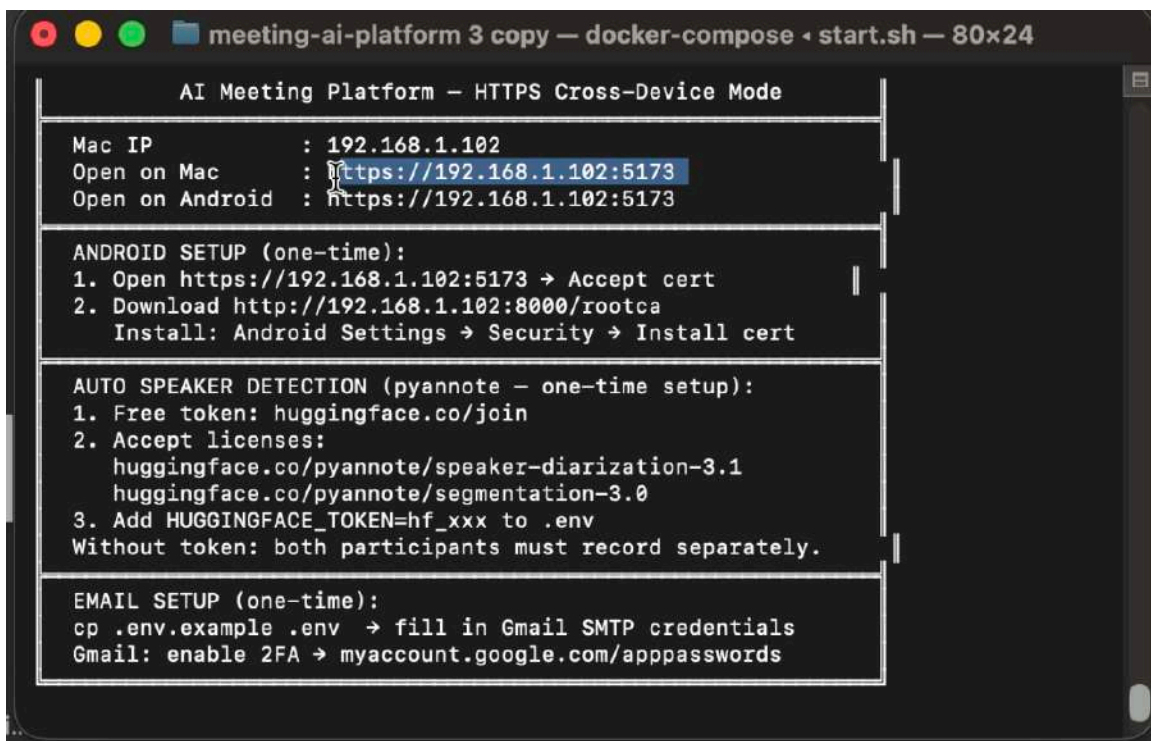


Figure 11.3: All Service URLs Screenshot

Description

Terminal screenshot showing the URLs for the frontend (<http://localhost:3000>), backend API (<http://localhost:8000>), and LiveKit server (<http://localhost:7880>).

Purpose

To show the user which URLs to open to access different parts of the system.

11.2 Landing Page

The landing page is the first page a user sees when they visit the system. It explains what the platform does and has a button to create a new meeting or join an existing one.

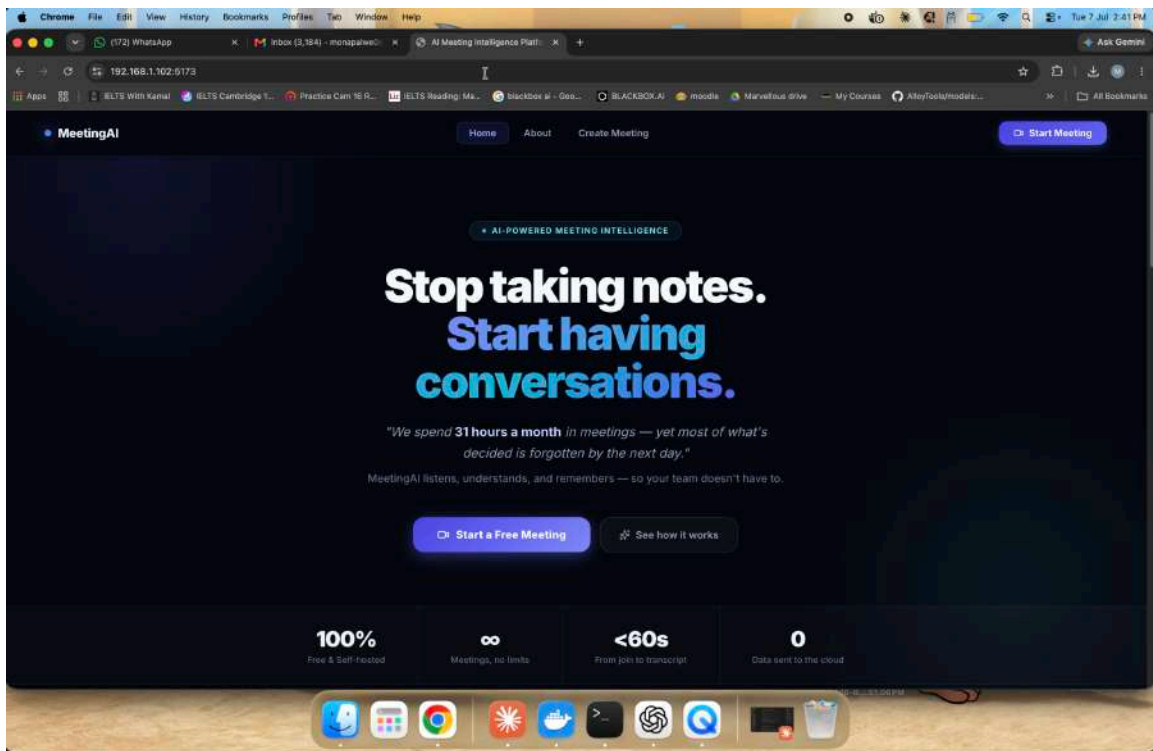


Figure 11.4: Landing Page — Hero Section Screenshot

Description

Browser screenshot of the landing page showing the title "AI Meeting Intelligence Platform", a short description, and two buttons: "Create Meeting" and "Join Meeting".

Purpose

To show the entry point of the application.

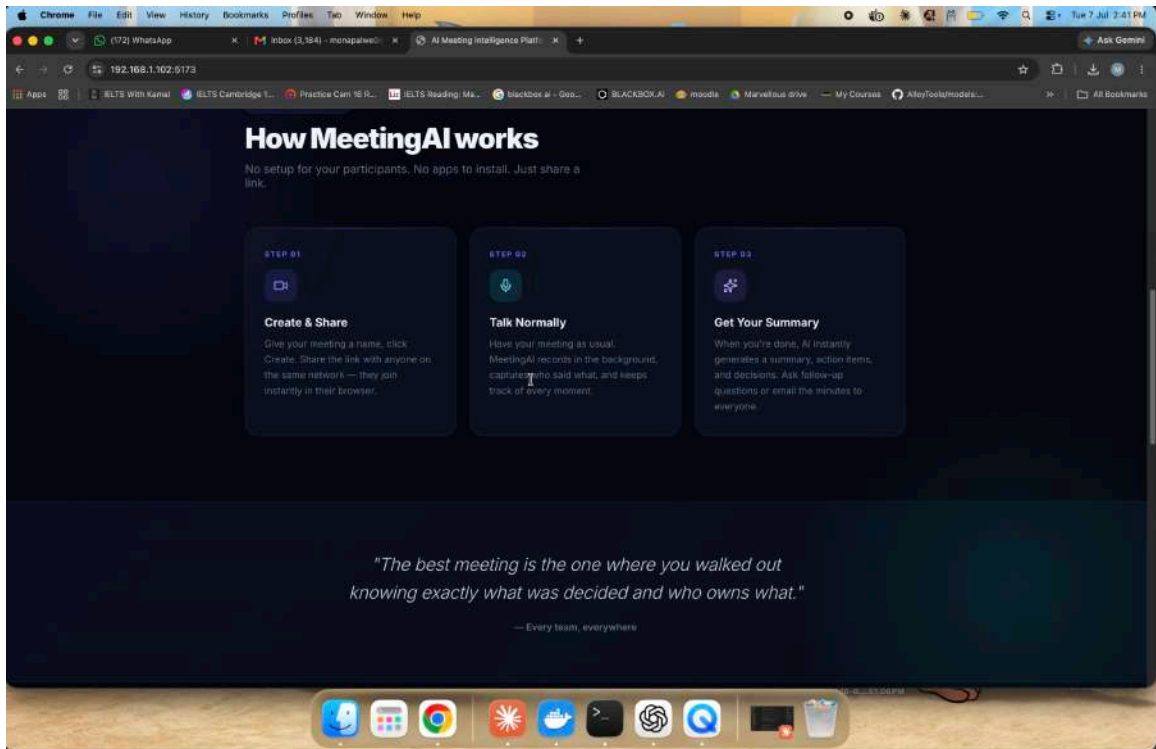


Figure 11.5: Landing Page — How It Works Screenshot

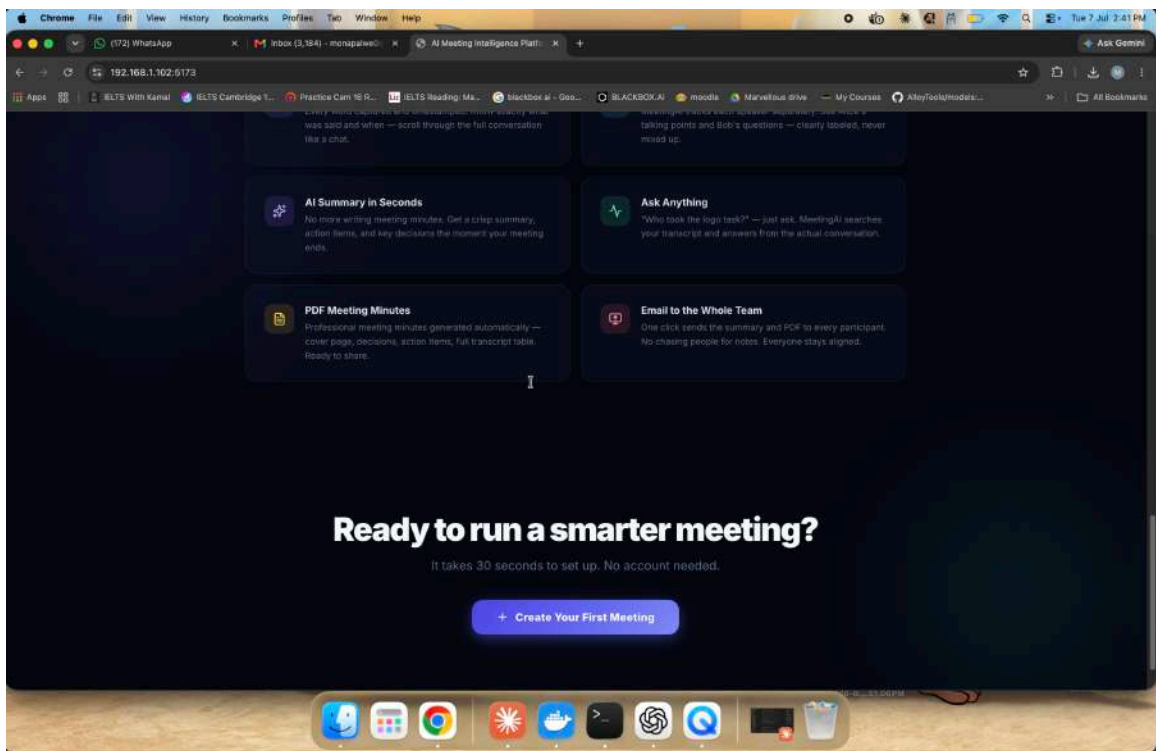


Figure 11.6: Landing Page — Feature Highlights Screenshot

11.3 Create Meeting

The host clicks “Create Meeting” and fills in a short form with the meeting title and optional description. When submitted, the system creates a new meeting record in the database and generates a unique join link.

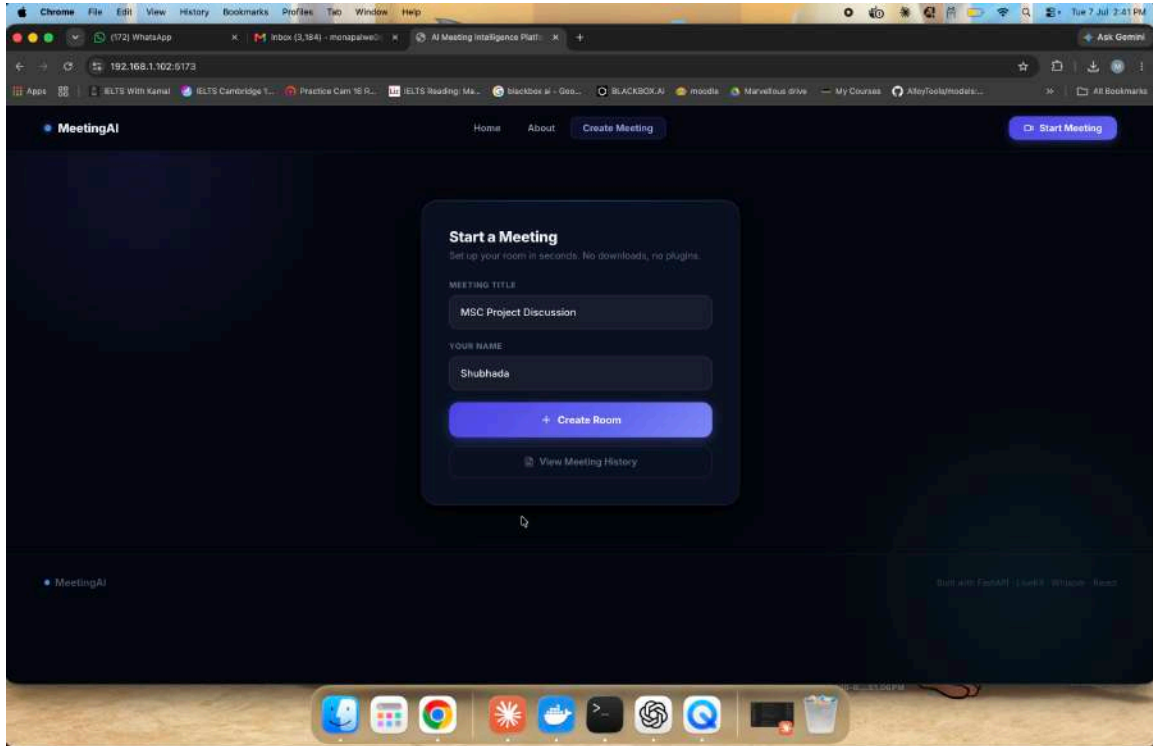


Figure 11.7: Create Meeting Form Screenshot

Description

Browser screenshot of a simple form with fields for "Meeting Title" and "Description (optional)". A blue "Create Meeting" button appears below the form.

Purpose

To show how a new meeting is created.

11.4 Invite Sharing

After creating the meeting, the host is shown the join link. They can copy this link and share it via WhatsApp, email, or any messaging app. Anyone who opens the link on the same network is taken directly to the meeting join page.

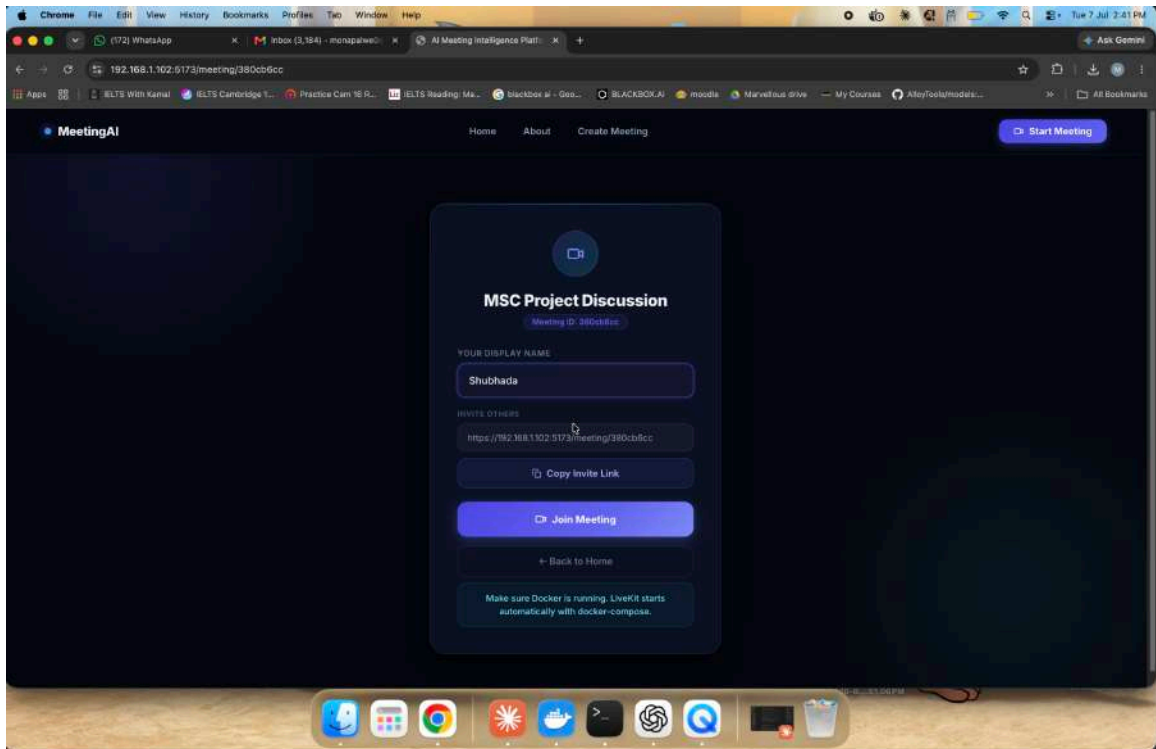


Figure 11.8: Meeting Join Page Screenshot

Description

Browser screenshot showing the meeting join page with the meeting title, a text field for entering the participant's name, and a "Join Meeting" button.

Purpose

To show how a participant joins the meeting using the shared link

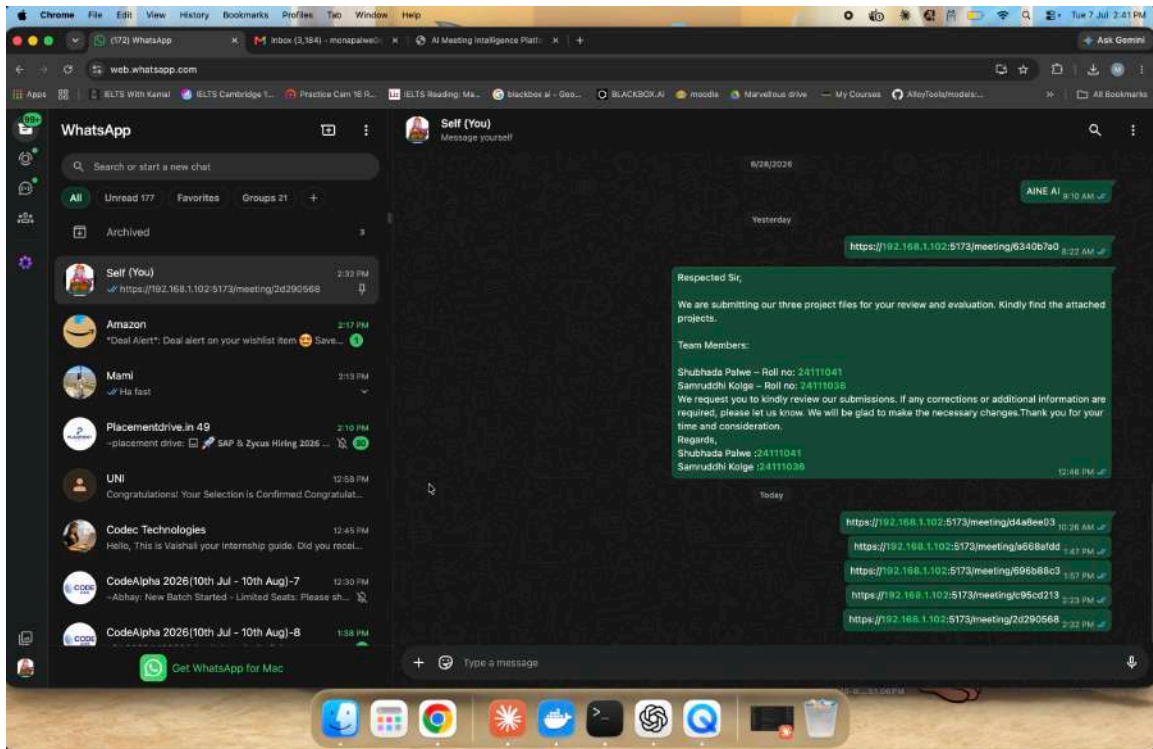


Figure 11.9: Invite Shared via WhatsApp Screenshot

Description

Mobile screenshot of WhatsApp showing a chat where the meeting link has been pasted and shared with another contact.

Purpose

To demonstrate how easy it is to share the meeting invite with others.

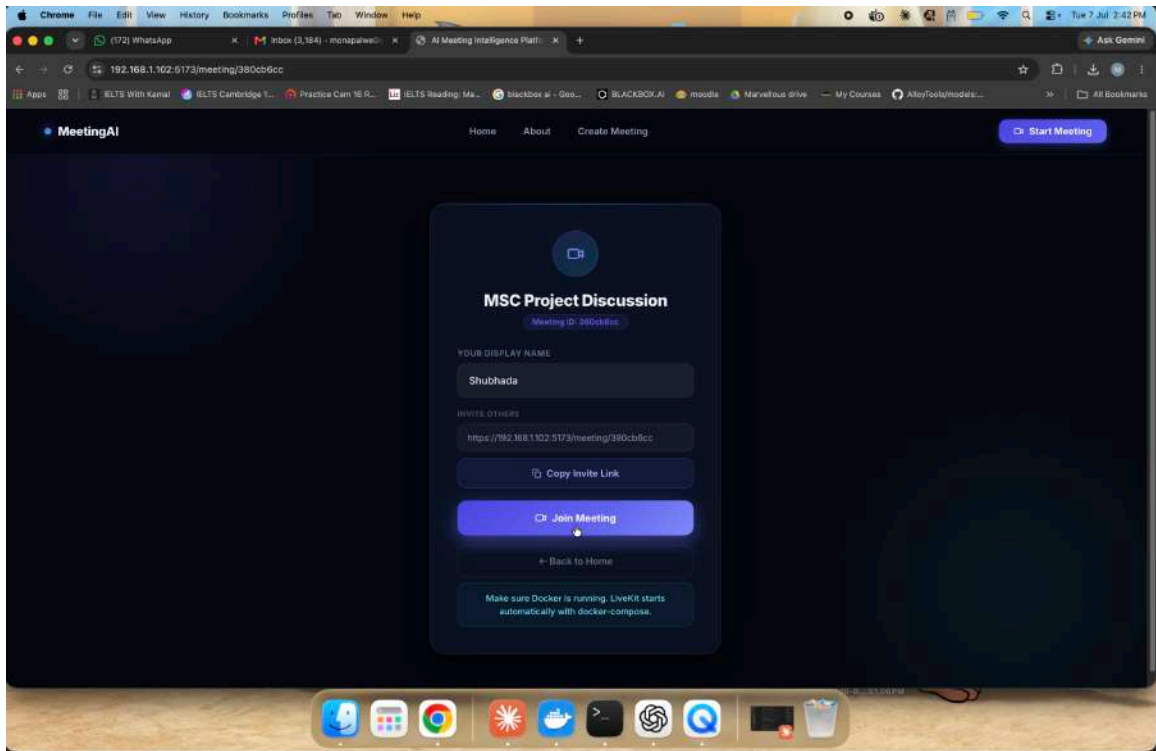


Figure 11.10: Second Participant Join Page Screenshot

11.5 Meeting Room

Once participants join, they appear in the meeting room. Each participant's video tile is shown on screen. The host sees controls at the bottom of the screen for muting, camera, screen sharing, and recording.

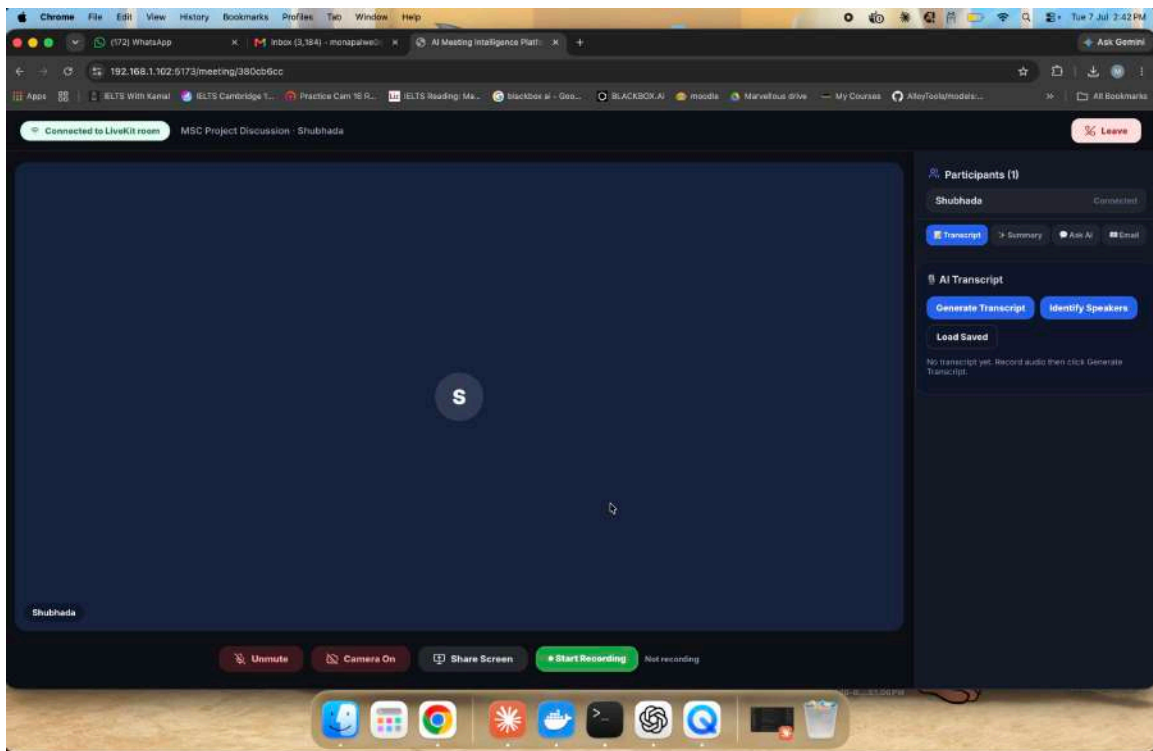


Figure 11.11: Meeting Room — One Participant Screenshot

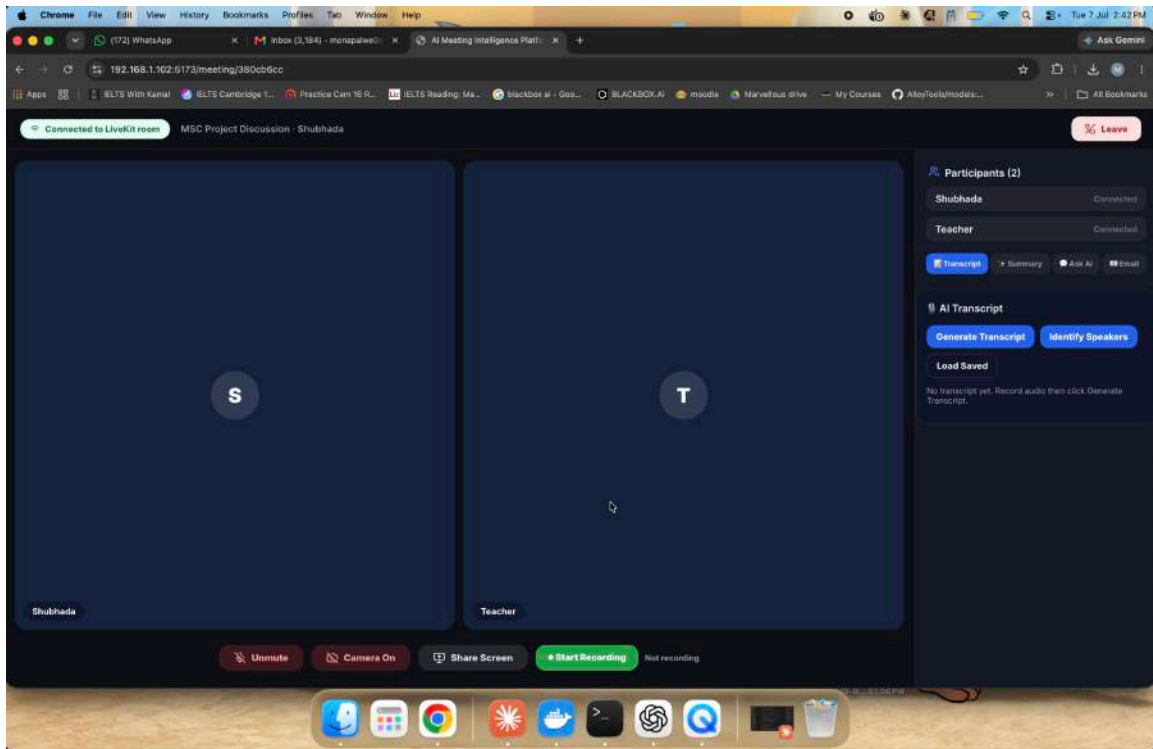


Figure 11.12: Meeting Room — Two Participants Screenshot

Description

Browser screenshot on a second device showing the same join page, demonstrating that multiple participants can join from different devices.

Purpose

To show that participants on separate devices can join the same meeting.

11.6 Active Speaker and Screen Sharing

The system automatically detects the active speaker (the person currently talking) and highlights their video tile with a coloured border. Screen sharing is also supported — participants can share their screen for presentations.

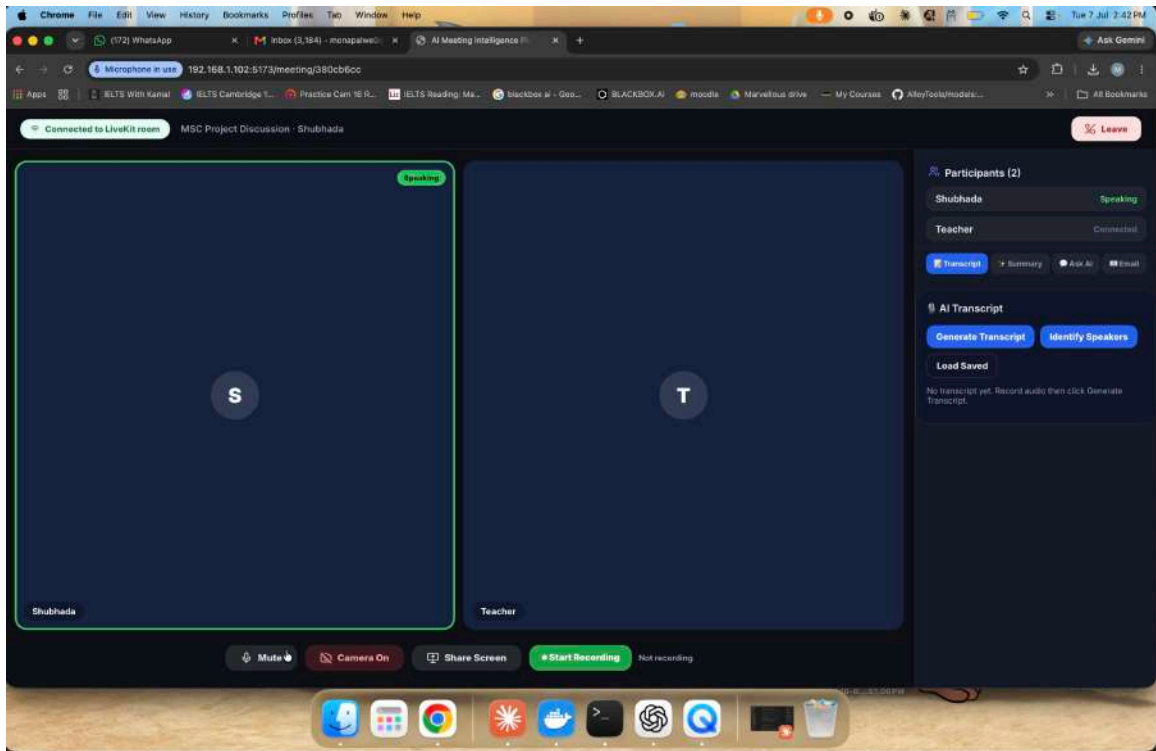


Figure 11.13: Active Speaker Detection Screenshot

Description

Browser screenshot showing the meeting room with the active speaker's video tile highlighted with a coloured border around it.

Purpose

To demonstrate the active speaker detection feature.

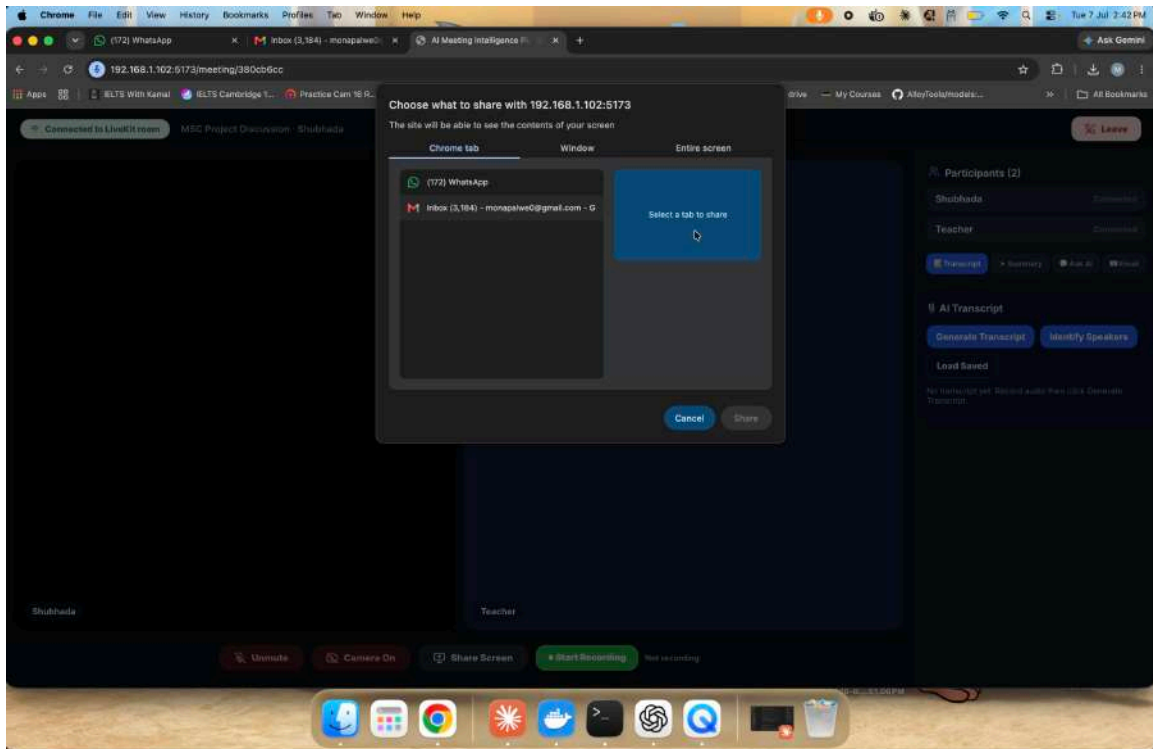


Figure 11.14: Screen Sharing Dialog Screenshot

Description

Browser screenshot showing the screen selection dialog that appears when a participant clicks the screen share button.

Purpose

To show how screen sharing is initiated.

11.7 Audio Recording

The host can start recording at any time during the meeting by clicking the Record button. A red indicator appears to show that recording is active. When the host clicks Stop Recording, the audio is saved and transcription begins automatically.

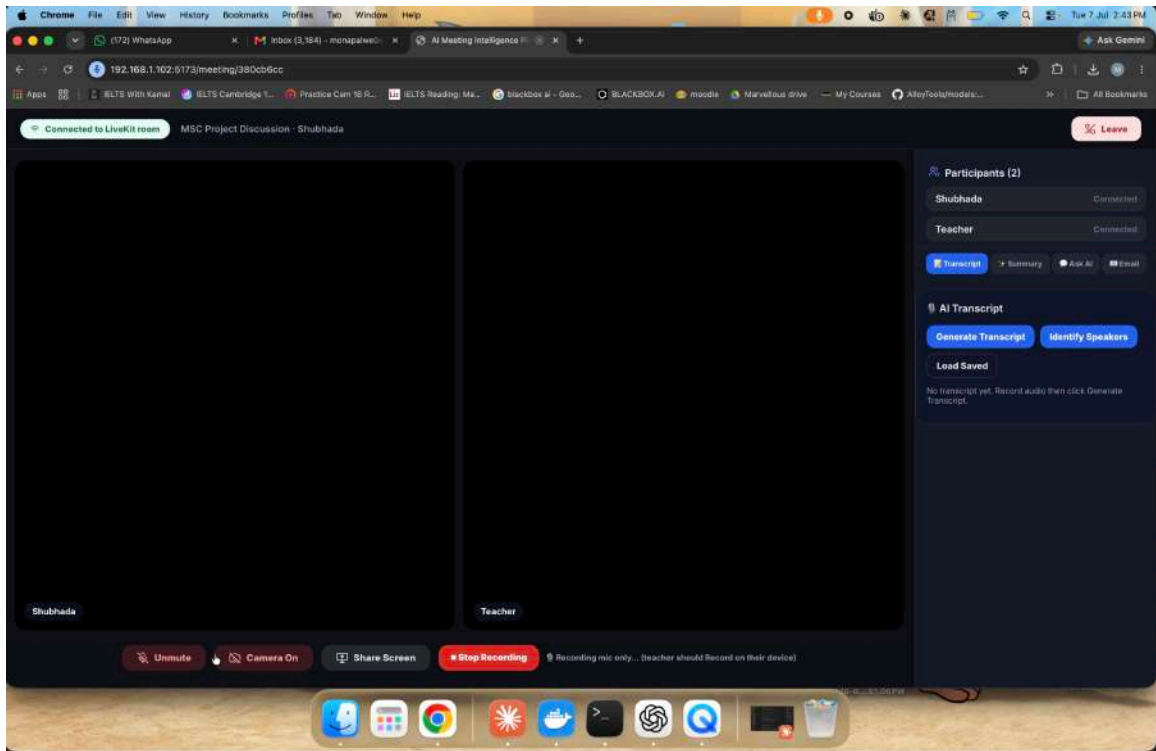


Figure 11.15: Recording Active Screenshot

Description

Browser screenshot of the meeting room showing a red "Recording" badge on the control bar to indicate that recording is in progress.

Purpose

To show the recording indicator that lets all participants know the meeting is being recorded.

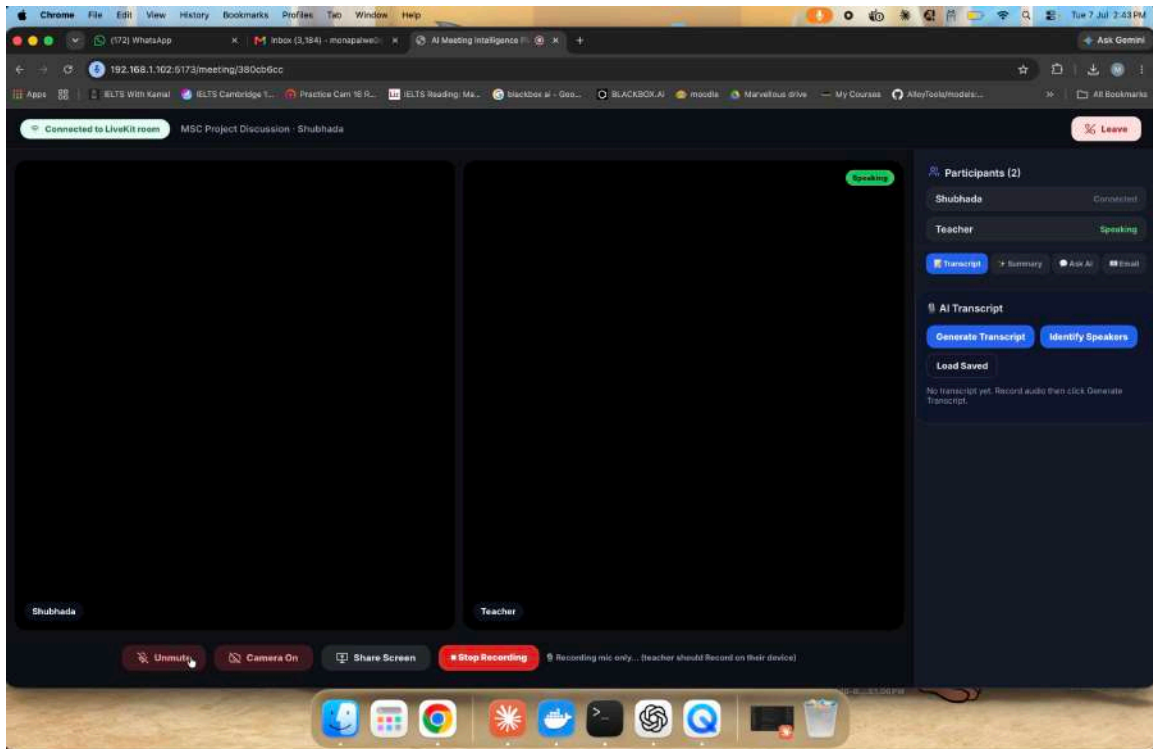


Figure 11.16: Teacher Recording View Screenshot

Description

Browser screenshot showing the host's view of the meeting room with the Start/Stop Recording button and the recording status.

Purpose

To show the recording controls available to the host.

11.8 Transcription

After the recording is stopped, the system processes the audio file through the four-stage transcription pipeline. A progress message appears while processing is happening. When complete, the full transcript appears in the Transcript tab, showing each speaker's name and what they said.

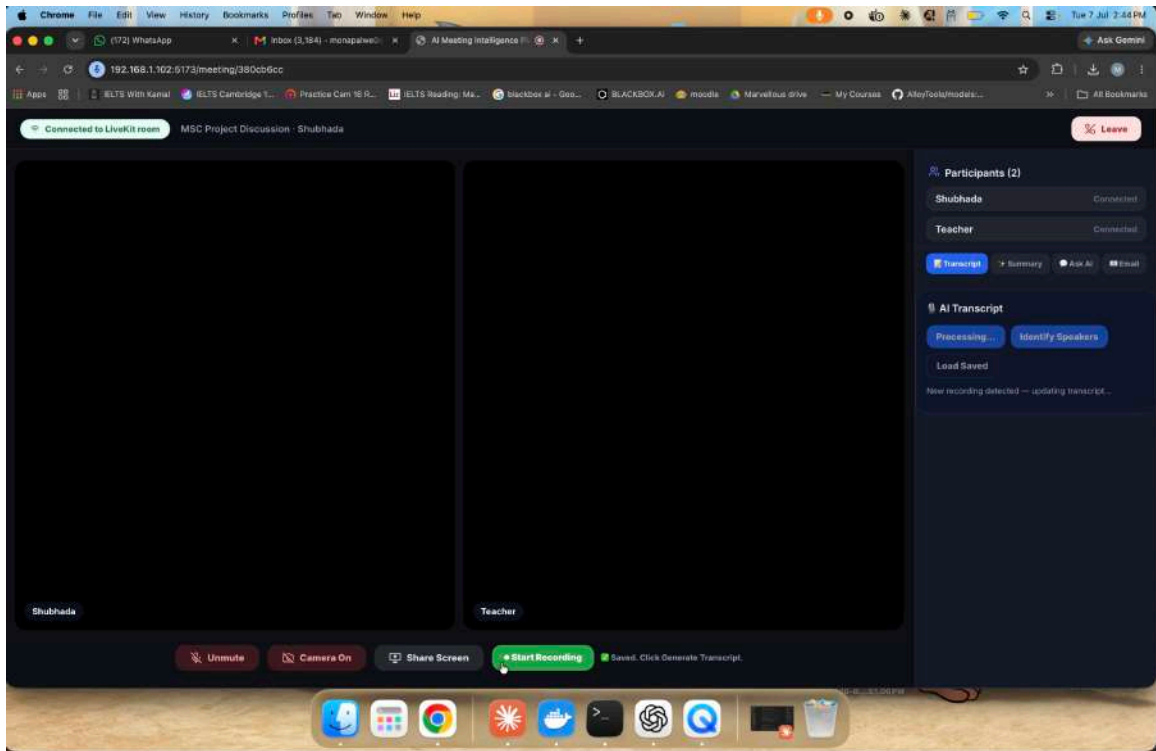


Figure 11.17: Transcription In Progress Screenshot

Description

Browser screenshot showing a loading message: "Processing transcript... This may take a minute." in the Transcript tab.

Purpose

To show the user that transcription is running in the background.

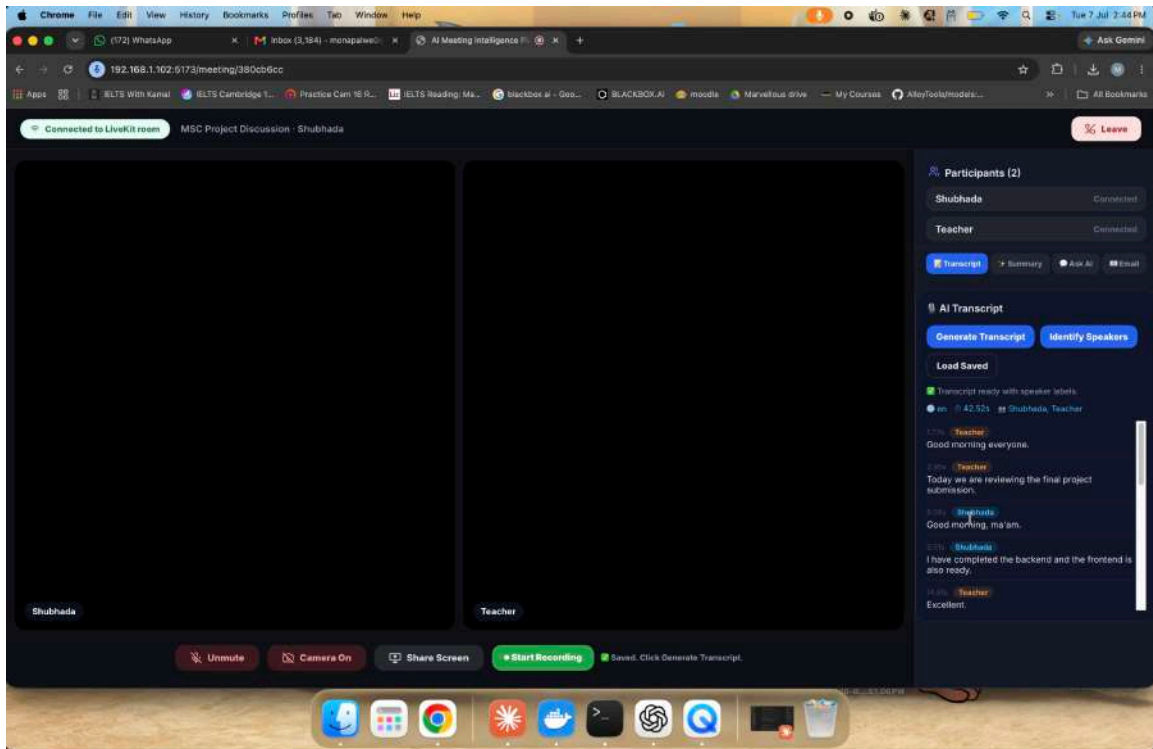


Figure 11.18: Completed Transcript Screenshot

Description

Browser screenshot of the completed transcript showing turns labelled "Speaker 1" and "Speaker 2" with timestamps and the text of what each person said.

Purpose

To show the final output of the transcription module

11.9 AI Summary

The AI Summary tab shows buttons to generate different parts of the meeting analysis. The user can click "Generate Summary" to get a written summary of the meeting. The summary is structured with key points, decisions made, and action items.

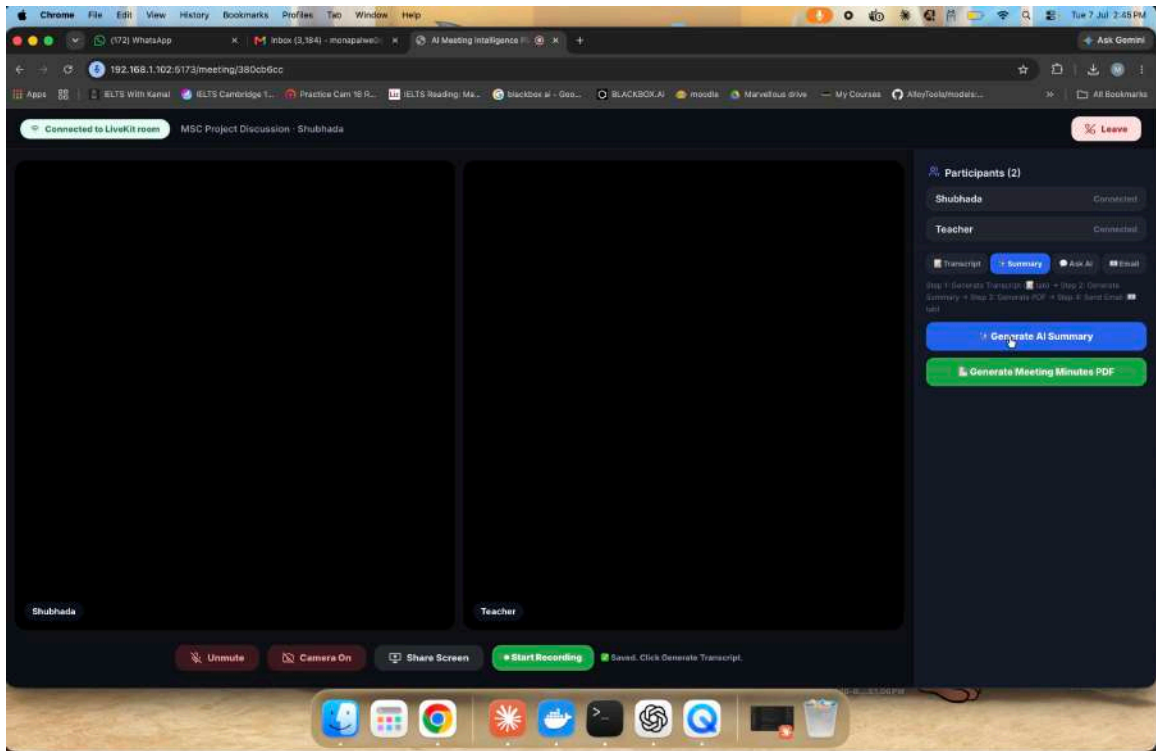


Figure 11.19: Summary Tab — Buttons Active Screenshot

Description

Browser screenshot of the Summary tab showing three buttons: "Generate Summary", "Generate Action Items", and "Get Key Decisions".

Purpose

To show the options available to the user in the AI Summary tab.

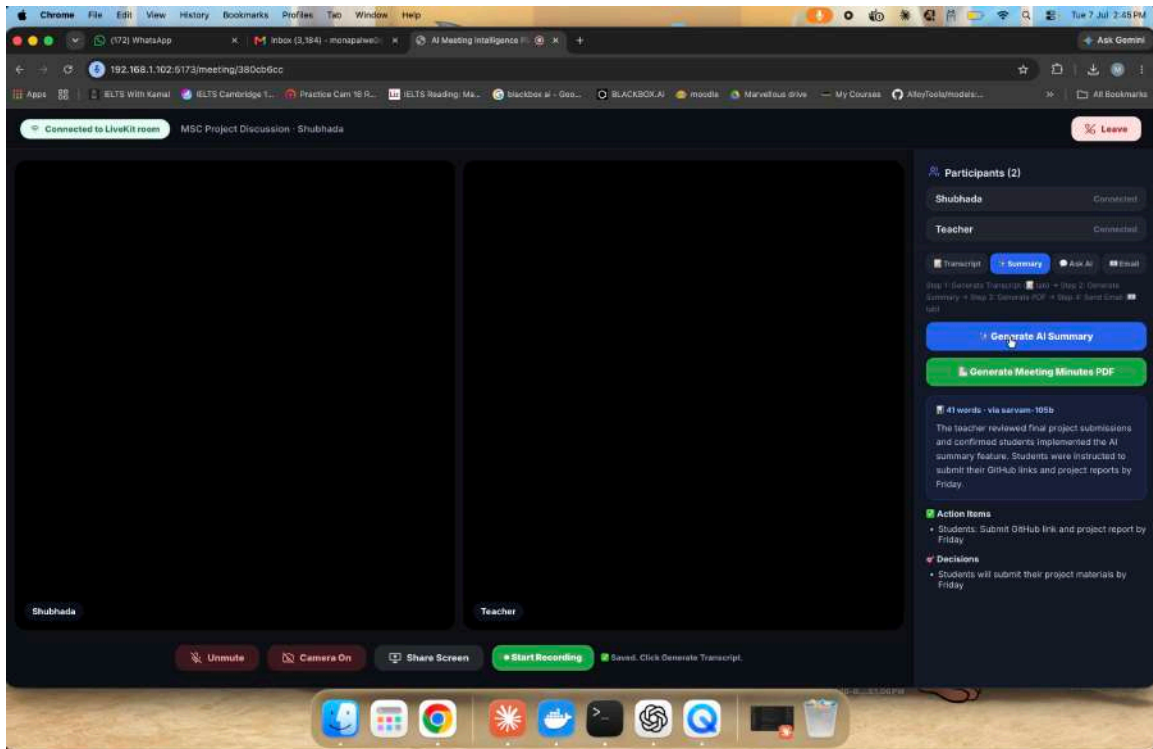


Figure 11.20: Generated AI Summary Screenshot

Description

Browser screenshot showing the AI-generated meeting summary with sections for Overview, Key Points Discussed, Decisions Made, and Action Items.

Purpose

To show the actual output of the AI summary feature.

11.10 Email Digest

The user can type an email address in the Email tab and send a PDF of the meeting minutes directly from the platform. The PDF includes the meeting title, date, summary, and full transcript.

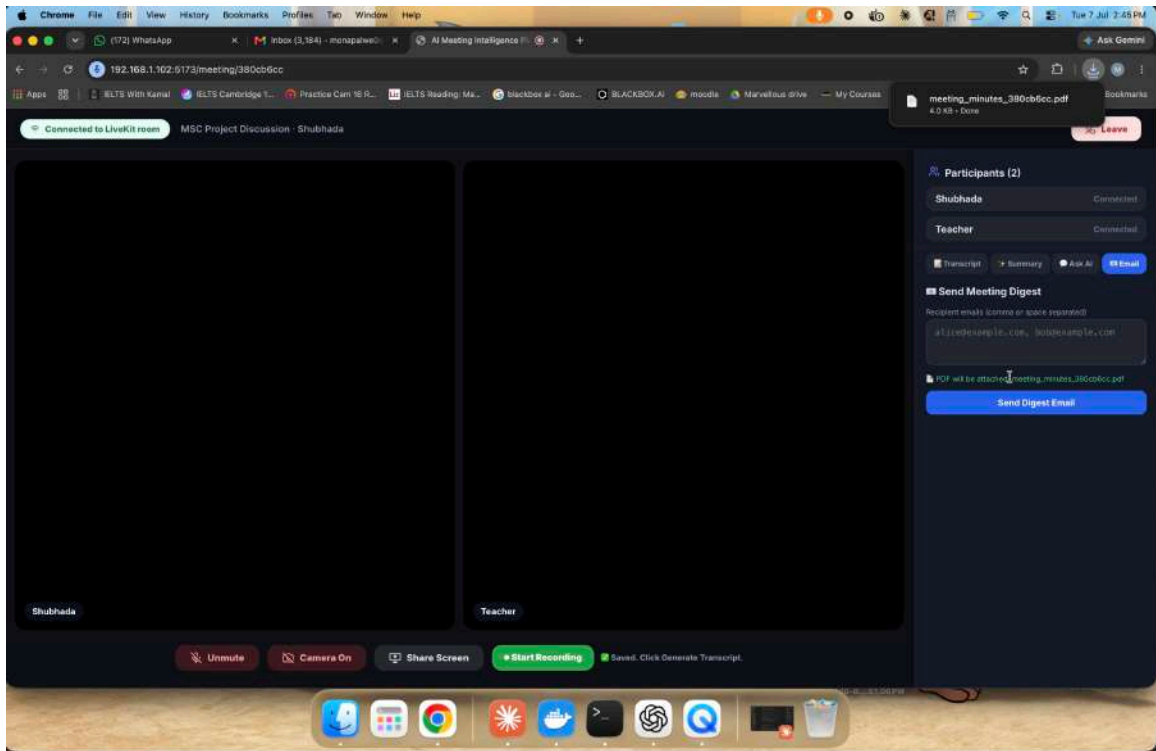


Figure 11.21: Email Tab — Recipient Input Screenshot

Description

Browser screenshot of the Email tab with an input field for entering the recipient's email address.

Purpose

To show how the user prepares to send the email digest.

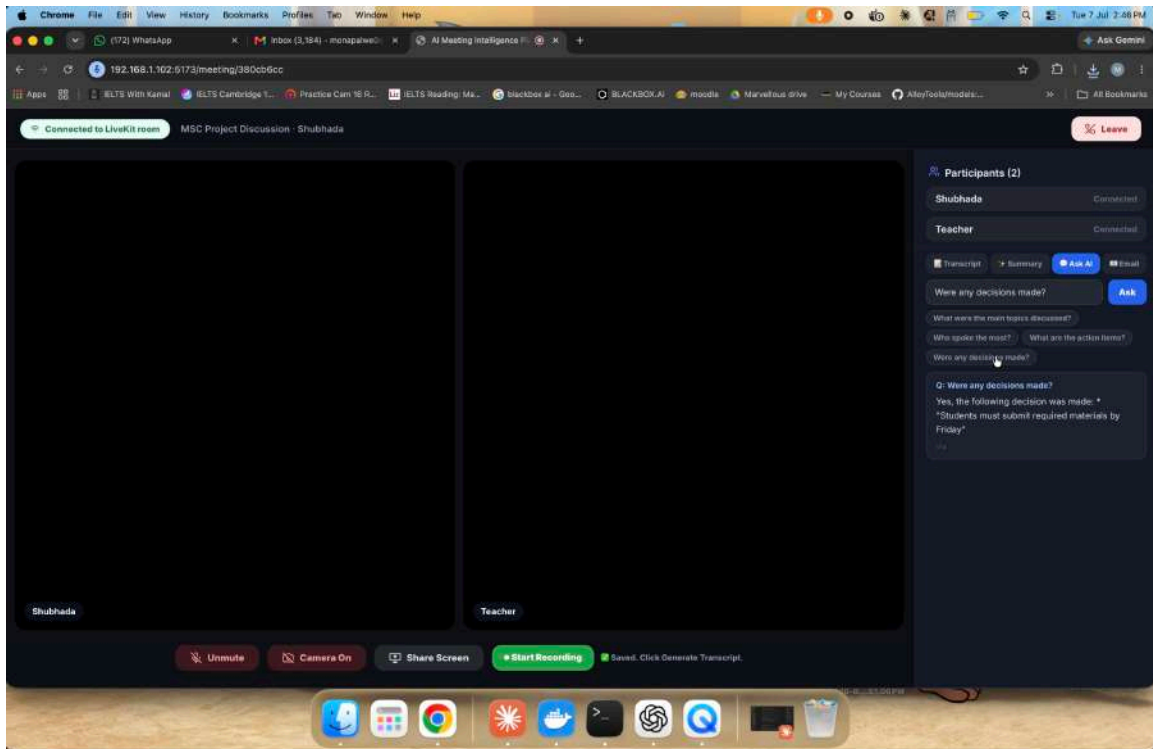


Figure 11.22: Email Ready to Send Screenshot

Description

Browser screenshot showing the Email tab after an email address has been entered, with a "Send Email" button ready to click.

Purpose

To show the email sending confirmation step.

11.11 Ask AI (Q&A)

The Ask AI tab allows users to type any question about the meeting and get an AI-written answer based on the transcript. This uses Retrieval-Augmented Generation (RAG) — the system searches the transcript for relevant sections and gives the language model only those parts to answer from.

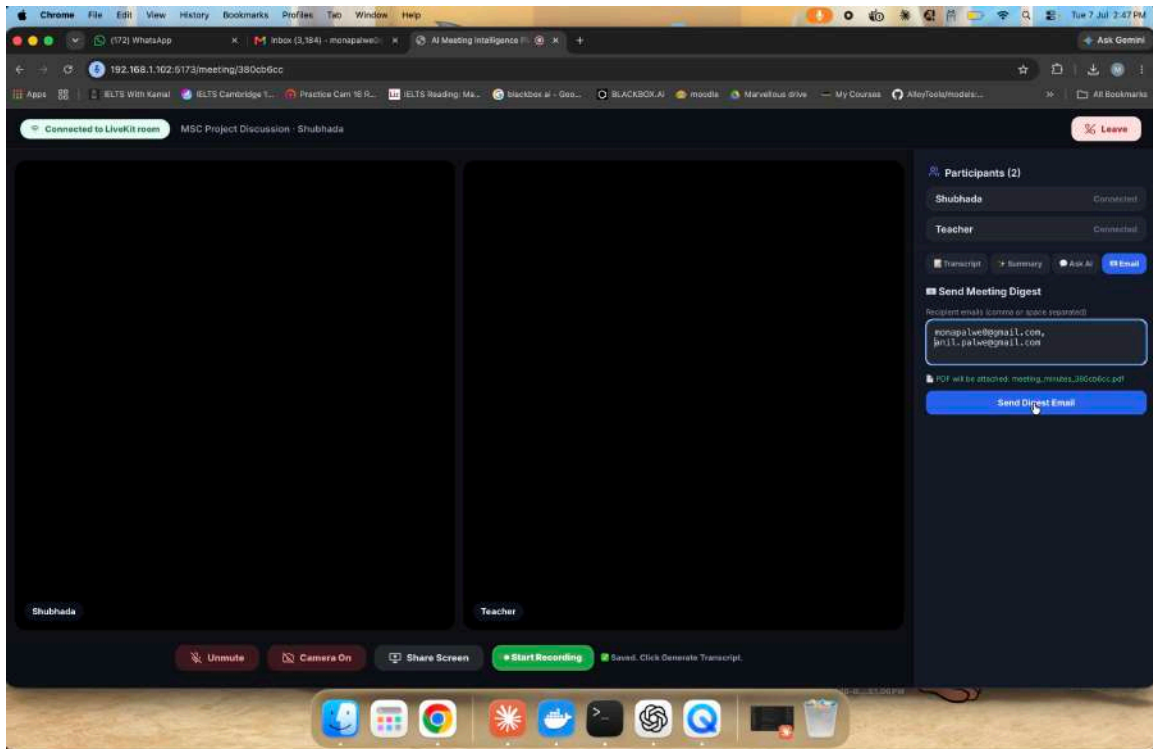


Figure 11.23: Ask AI Q&A Panel Screenshot

Description

Browser screenshot of the Ask AI tab showing a question input field and an example AI answer to the question "What was decided about the next meeting date?".

Purpose

To demonstrate the Q&A feature that lets users ask natural language questions about the meeting.

11.12 Transcription Pipeline Architecture

The following three diagrams explain in detail how the transcription, summarisation, and Q&A pipelines work inside the backend.

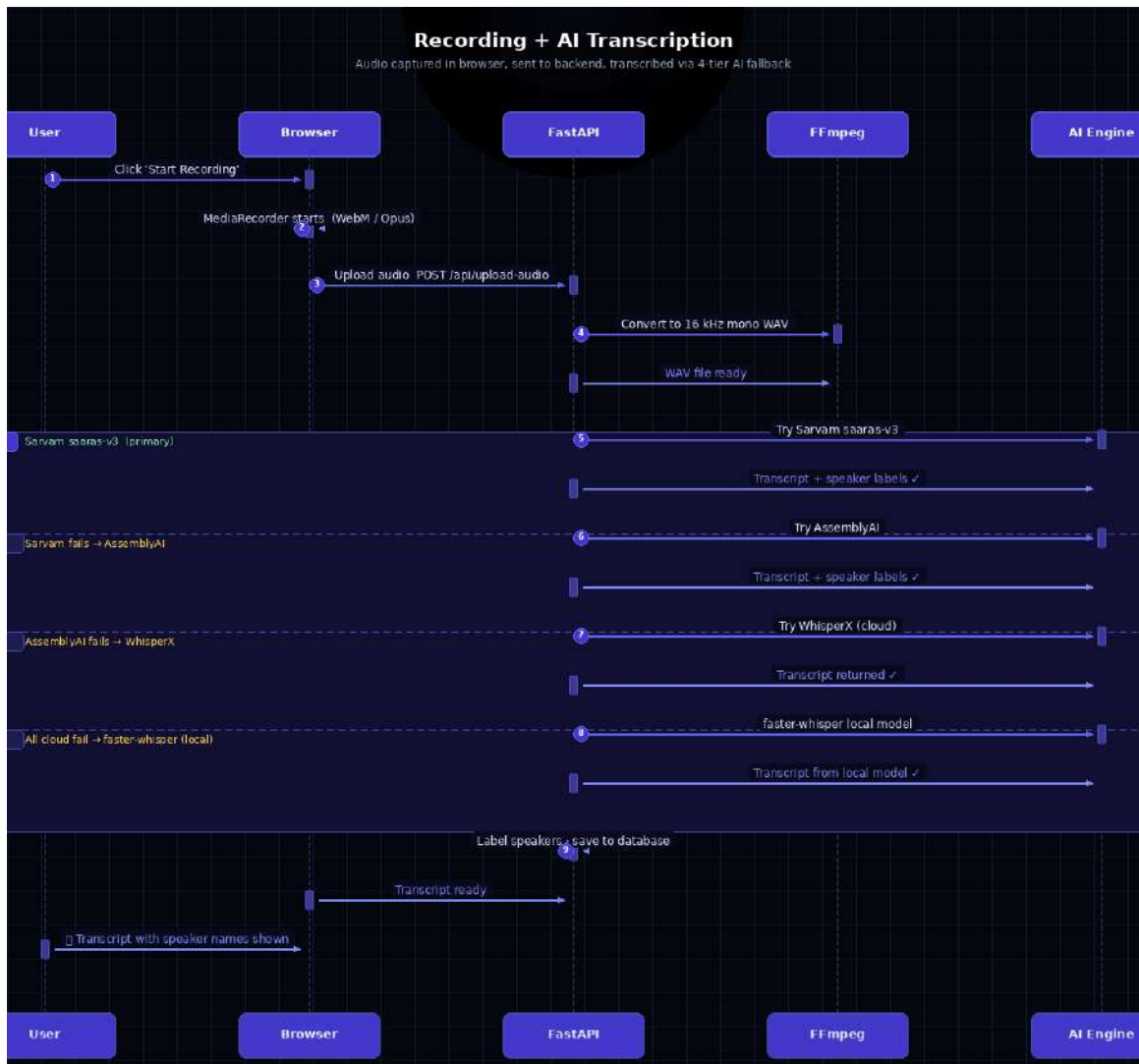


Figure 11.24: Transcription Pipeline Flow Diagram

Description

A flowchart showing the four-stage transcription process: Stage 1 Sarvam AI, Stage 2 AssemblyAI, Stage 3 WhisperX, Stage 4 faster-whisper. Each stage shows the input (audio file), the output (transcript JSON), and the error path that triggers the next stage.

Purpose

To explain in detail how the transcription fallback system works.

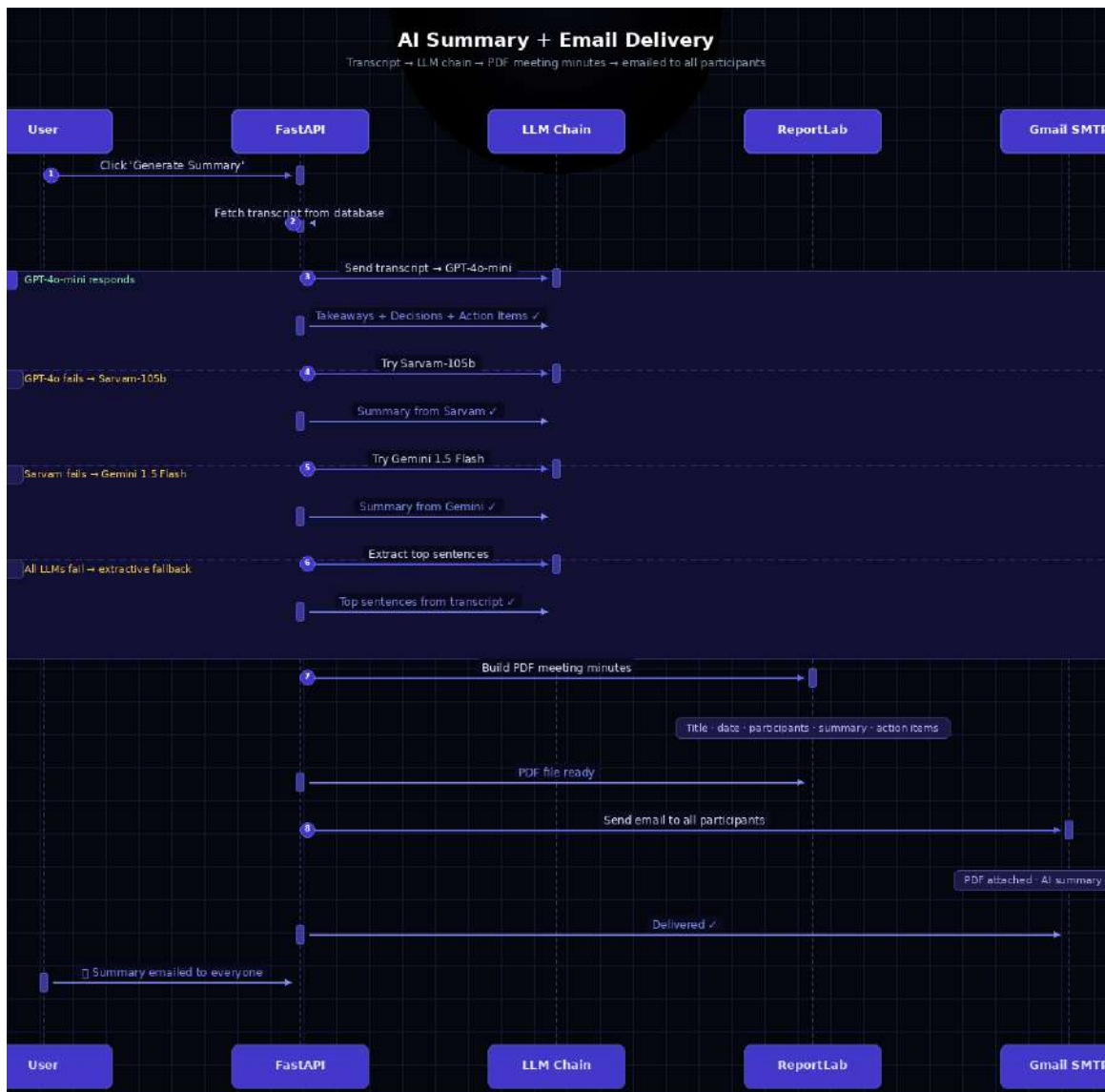


Figure 11.25: AI Summarisation Flow Diagram

Description

A flowchart starting from the completed transcript. Shows how the text is chunked, sent to the LLM with a prompt, and returned as a structured summary with sections for Overview, Decisions, and Action Items.

Purpose

To explain how the AI summary is generated from the transcript.

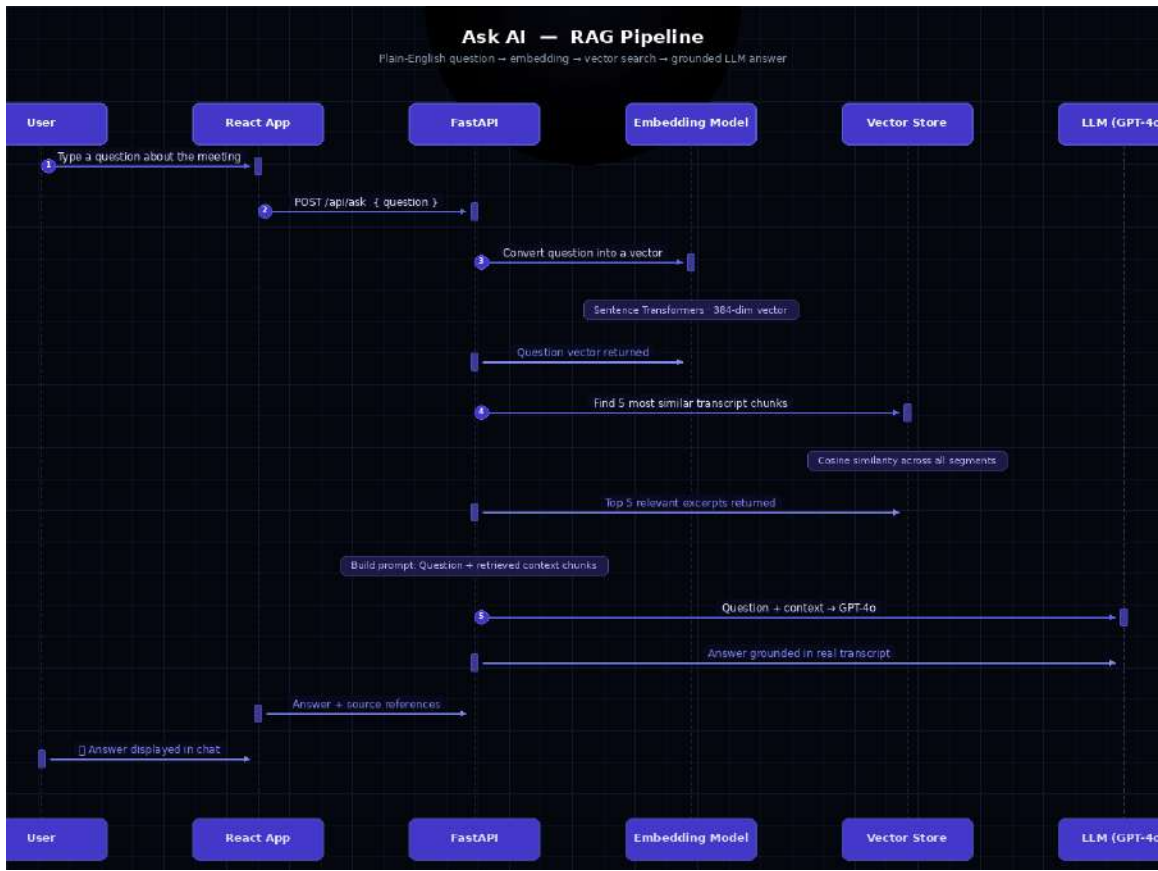


Figure 11.26: RAG Q&A Flow Diagram

Description

A flowchart showing the RAG (Retrieval-Augmented Generation) process: the user question is embedded → similar transcript chunks are retrieved → the relevant chunks and the question are sent to the LLM → the answer is returned to the user.

Purpose

accurately.

CHAPTER 12: DATABASE DESIGN

The system uses a SQLite database with four main tables: Meetings, Participants, Transcripts, and Summaries. SQLite was chosen because it is a lightweight file-based database that requires no installation or configuration.

12.1 Table Descriptions

The Meetings table stores the meeting ID, title, description, status (scheduled/active/completed), start time, and end time. The Participants table stores participant names, their assigned speaker labels, and which meeting they belong to. The Transcripts table stores the full transcript text and a JSON object with each speaker's turns. The Summaries table stores the AI-generated summary text, key decisions, and action items.

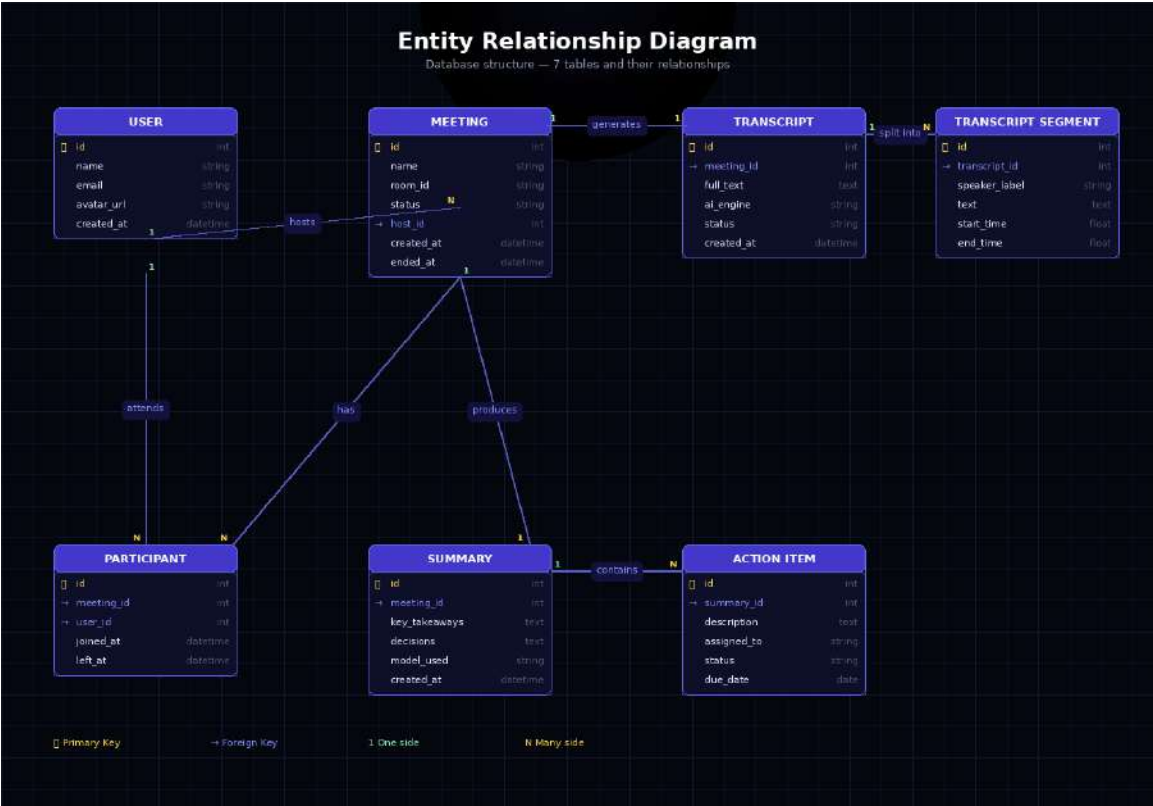


Figure 12.1: Entity-Relationship (ER) Diagram

Description

An ER diagram showing four entities: Meeting, Participant, Transcript, and Summary. Lines connecting them show relationships: one Meeting has many Participants (1 to many); one Meeting has one Transcript (1 to 1); one Meeting has one Summary (1 to 1).

Purpose

To show the structure of the database and how the different tables relate to each other.

CHAPTER 13: API DESIGN

13.1 API Endpoints

The backend exposes a REST API built with FastAPI. The main endpoints are:

Method	Endpoint	Purpose
POST	/meetings	Create a new meeting
GET	/meetings/{id}	Get meeting details
POST	/meetings/{id}/start-	Start server-side recording
POST	/meetings/{id}/stop-recording	Stop recording and trigger transcription
GET	/meetings/{id}/transcript	Get the completed transcript
GET	/meetings/{id}/summary	Get AI summary (generates if not exists)
POST	/meetings/{id}/ask	Ask a question about the meeting
POST	/meetings/{id}/send-email	Send PDF minutes by email
GET	/health	Check if all services are running

API Routes Map — FastAPI Endpoints			
All routes grouped by domain - method - request body - response shape			
METHOD	ENDPOINT	REQUEST BODY	RESPONSE
POST	/api/meeting/create	name, host_name	room_id, token, join_url
POST	/api/meeting/join	room_id, user_name	token, participant_id
GET	/api/meeting/{room_id}	-	id, name, status, participants[]
DELETE	/api/meeting/{room_id}	-	status: cancelled
POST	/api/recording/upload	audio_file (binary), meeting_id	file_id, duration_s, format
GET	/api/recording/{meeting_id}	-	url, format, size_bytes, duration_s
POST	/api/transcript/generate	meeting_id	transcript_id, engine_used, status
GET	/api/transcript/{meeting_id}	-	full_text, segments[], speakers[]
POST	/api/summary/generate	meeting_id	summary_id, model_used, status
GET	/api/summary/{meeting_id}	-	key_takeaways, decisions, action_items[]
POST	/api/ask	meeting_id, question	answer, confidence, source_segments[]
POST	/api/email/send	meeting_id, recipients[]	status, delivered_count, failed[]
GET	/health	-	status: ok, version, uptime_s

All protected routes require: Authorization: Bearer <JWT token>

Figure 13.1: API Routes Map Diagram

Description

A diagram showing all API routes grouped by function: Meeting Management routes, Recording routes, AI Processing routes, and Communication routes.

Purpose

To give a visual overview of the entire API surface of the backend.

13.2 Frontend Components

The React frontend is divided into six main components: LandingPage, CreateMeeting, JoinMeeting, MeetingRoom, PostMeetingDashboard, and AskAI. Each component handles a specific step in the user journey.

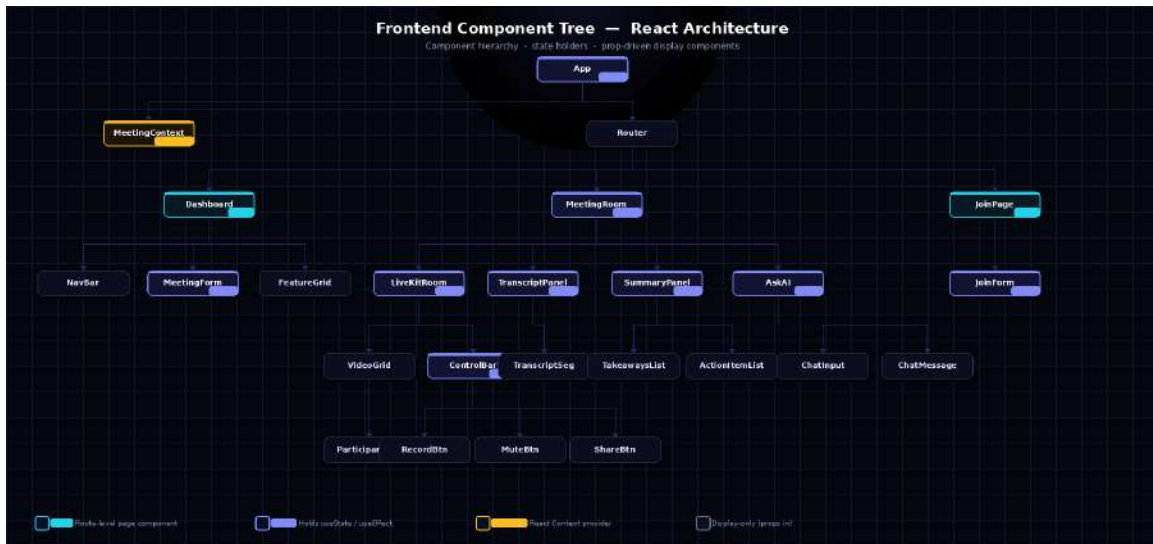


Figure 13.2: React Component Tree Diagram

Description

A tree diagram showing the React component hierarchy: *App* at the top, then *LandingPage*, *CreateMeeting*, *JoinMeeting* at the second level, *MeetingRoom* at the third level with child components (*VideoGrid*, *ControlBar*, *RecordingButton*), and *PostMeetingDashboard* at the fourth level with child components (*TranscriptTab*, *SummaryTab*, *EmailTab*, *AskAITab*).

Purpose

To show how the React components are structured and which component is the parent of which.

13.3 WebRTC Connection Flow

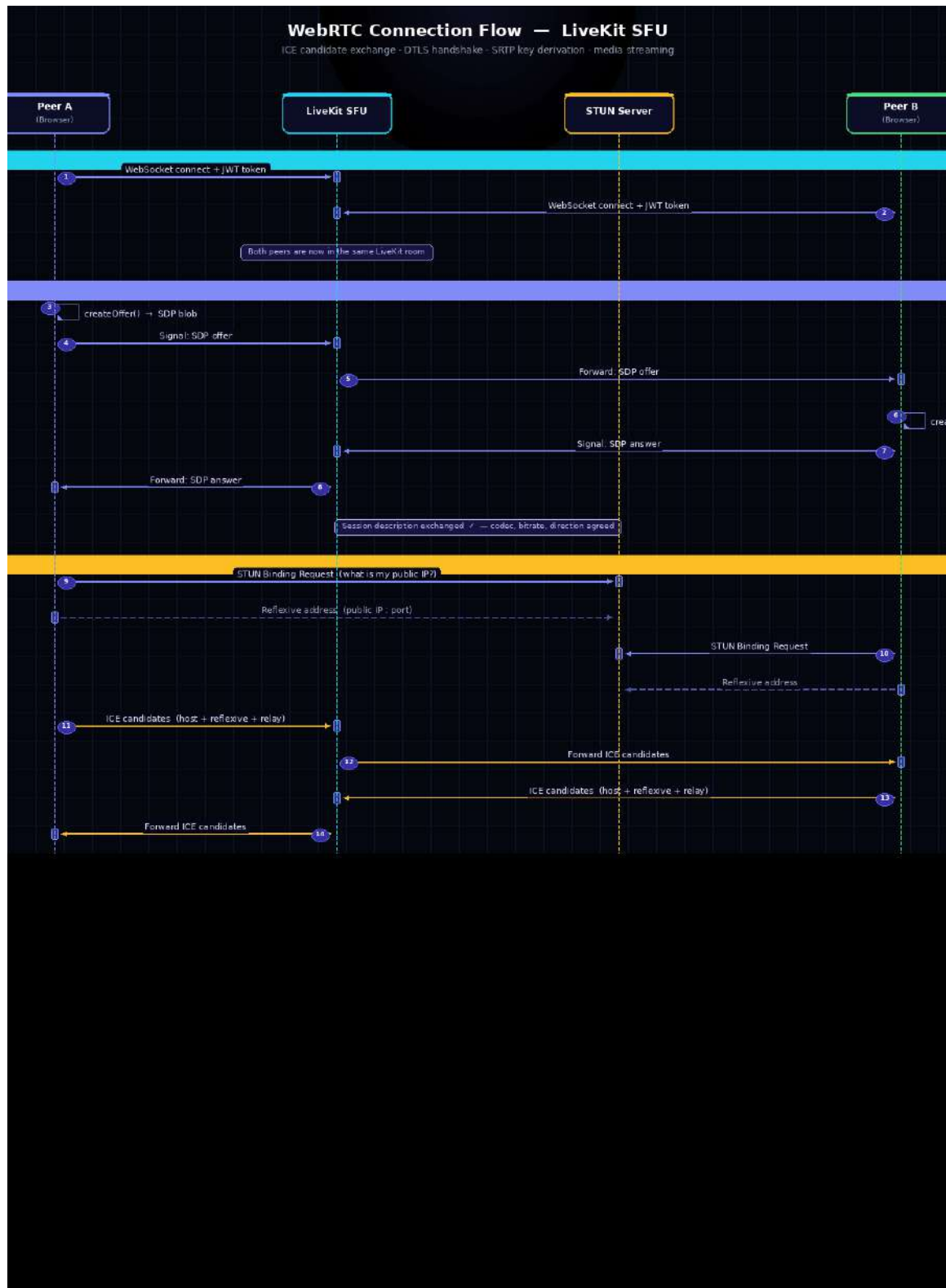


Figure 13.3: WebRTC Connection Flow Diagram

Description

A diagram showing how a browser connects to the LiveKit WebRTC server. Steps: browser requests a token from FastAPI → FastAPI generates a LiveKit access token → browser uses token to connect to LiveKit → LiveKit establishes peer-to-peer audio/video connections between all participants.

Purpose

To explain how the secure WebRTC connection is established between browsers and the LiveKit server.

CHAPTER 14: IMPLEMENTATION DETAILS

14.1 Folder Structure

The project is organised into two main folders: `/backend` (Python FastAPI) and `/frontend` (React TypeScript). A `docker-compose.yml` file at the root ties everything together.

Folder / File	Purpose
<code>backend/main.py</code>	FastAPI application entry point
<code>backend/models.py</code>	SQLite database models (SQLAlchemy)
<code>backend/transcription.py</code>	Four-stage transcription pipeline
<code>backend/summariser.py</code>	AI summary and Q&A logic
<code>backend/pdf_generator.py</code>	Meeting minutes PDF creator
<code>backend/email_sender.py</code>	SMTP email delivery
<code>frontend/src/pages/</code>	React page components
<code>frontend/src/components/</code>	Reusable UI components
<code>docker-compose.yml</code>	Starts all services together
<code>.env.example</code>	Template for environment variables

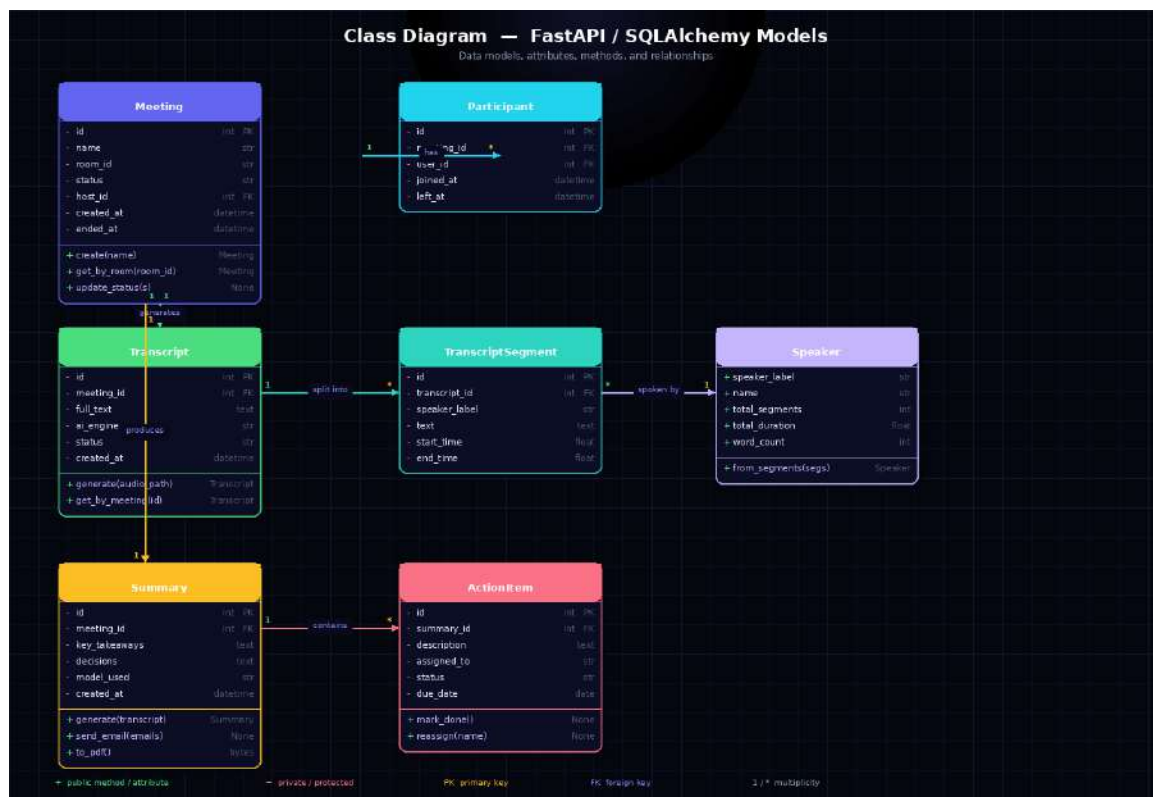


Figure 14.1: Class Diagram — Backend Services

Description

A UML class diagram showing the main Python classes in the backend: *MeetingService*, *TranscriptionService*, *SummarisationService*, *PDFGenerator*, and *EmailSender*. Shows public methods of each class and which classes call which.

Purpose

To show the object-oriented structure of the backend and the responsibilities of each service class.

14.2 Key Coding Patterns

The backend uses `async/await` extensively so that long-running tasks (like transcription) do not block the server from handling other requests. Background tasks are handled using FastAPI's `BackgroundTasks` feature.

The frontend uses React hooks (`useState`, `useEffect`, `useCallback`) for state management. The LiveKit SDK is wrapped in a custom hook (`useMeeting`) that manages the connection, audio/video tracks, and recording state.

14.3 Transcription Pipeline Code

The transcription pipeline is written as a single function that tries each engine in order. If an engine throws an exception or returns an empty result, it logs the error and moves to the next engine. The result from whichever engine succeeds is stored in the `Transcripts` table.

CHAPTER 16: TESTING

16.1 Testing Approach

Testing was done in two phases. Unit tests were written for the transcription pipeline to verify that each engine correctly handles success and failure cases. Integration testing was done by running the complete system end-to-end with two real participants.

16.2 Test Cases

Test ID	Test Description	Expected Result	Actual Result	Status
T01	Create meeting and get join link	Join link generated	Link generated correctly	Pass
T02	Two participants join meeting	Both video tiles appear	Both tiles appeared	Pass
T03	Start recording	Red recording badge shown	Badge appeared	Pass
T04	Stop recording and wait for transcript	Transcript generated	Transcript appeared in ~2 minutes	Pass
T05	Sarvam AI unavailable, fallback to AssemblyAI	AssemblyAI transcript returned	Fallback worked correctly	Pass
T06	All cloud engines unavailable, faster-whisper used	Local transcript returned	faster-whisper produced transcript	Pass
T07	Generate AI summary	Summary with sections produced	Summary generated correctly	Pass
T08	Ask AI a question	Relevant answer returned	Answer was accurate	Pass
T09	Send email with PDF	Email received in inbox	Email delivered in ~30 seconds	Pass
T10	Rejoin meeting after browser refresh	Participant rejoined successfully	Rejoined without issues	Pass

16.3 Performance Testing

A 10-minute meeting recording was processed using faster-whisper on a laptop with an Intel Core i7 CPU and 16 GB RAM. The transcription took approximately 4 minutes and 30 seconds to

complete. The AI summary was generated in under 30 seconds. PDF generation took less than 5 seconds.

CHAPTER 17: RESULTS

The system was tested successfully with a real meeting between two participants. All features worked as expected. The transcript correctly identified both speakers and the AI summary accurately captured the key points from the conversation. The PDF was generated automatically and delivered to the specified email address.

The following screenshots are taken from the results of the final test session and demonstrate that all features are working correctly in the deployed system.

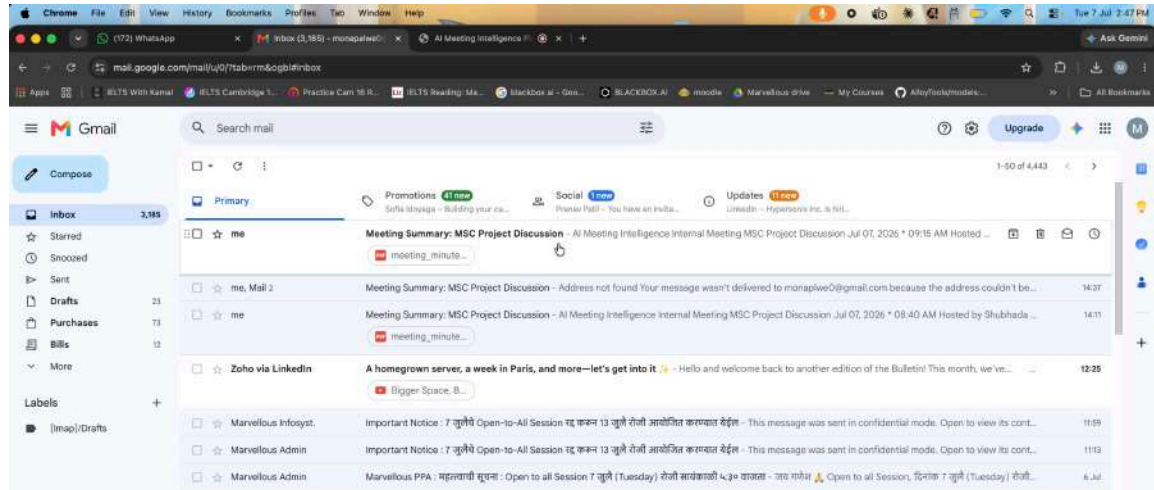


Figure 17.1: Results — Gmail Inbox with Email Received

Description

Screenshot of Gmail inbox showing the meeting digest email with subject "Meeting Minutes: [Meeting Title]" received from the platform.

Purpose

To prove end-to-end email delivery worked in the final test.

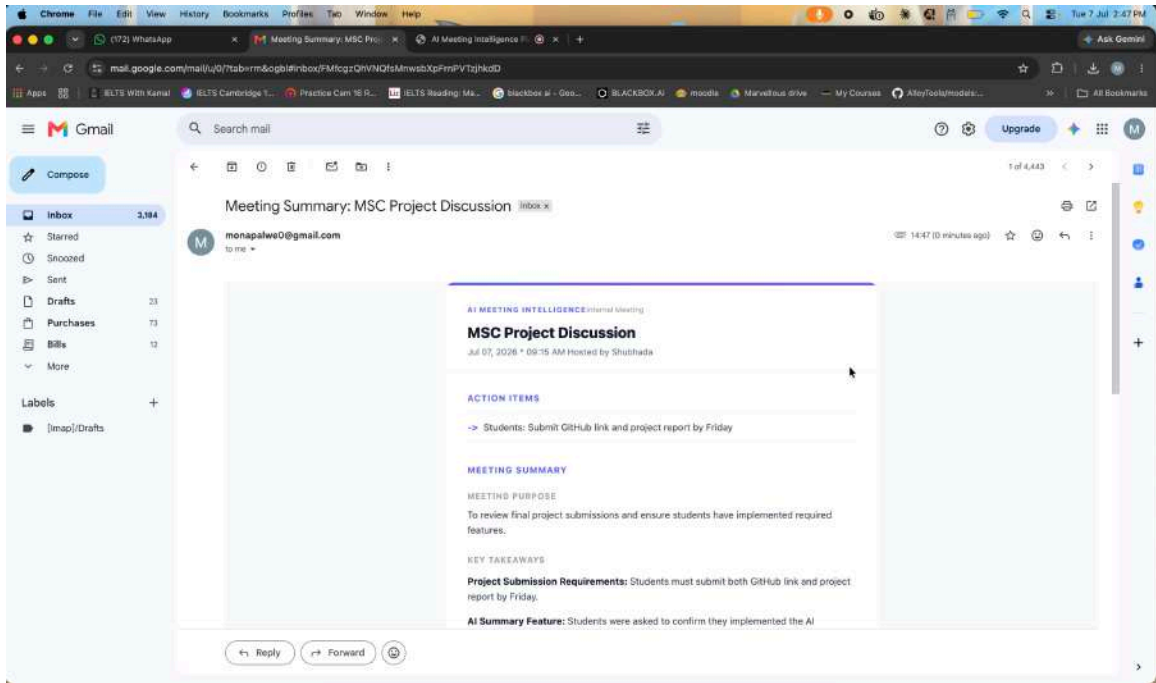


Figure 17.2: Results — Email Body Open

Description

Screenshot of the email opened in Gmail showing the AI summary text in the email body.

Purpose

To show the content of the delivered email.

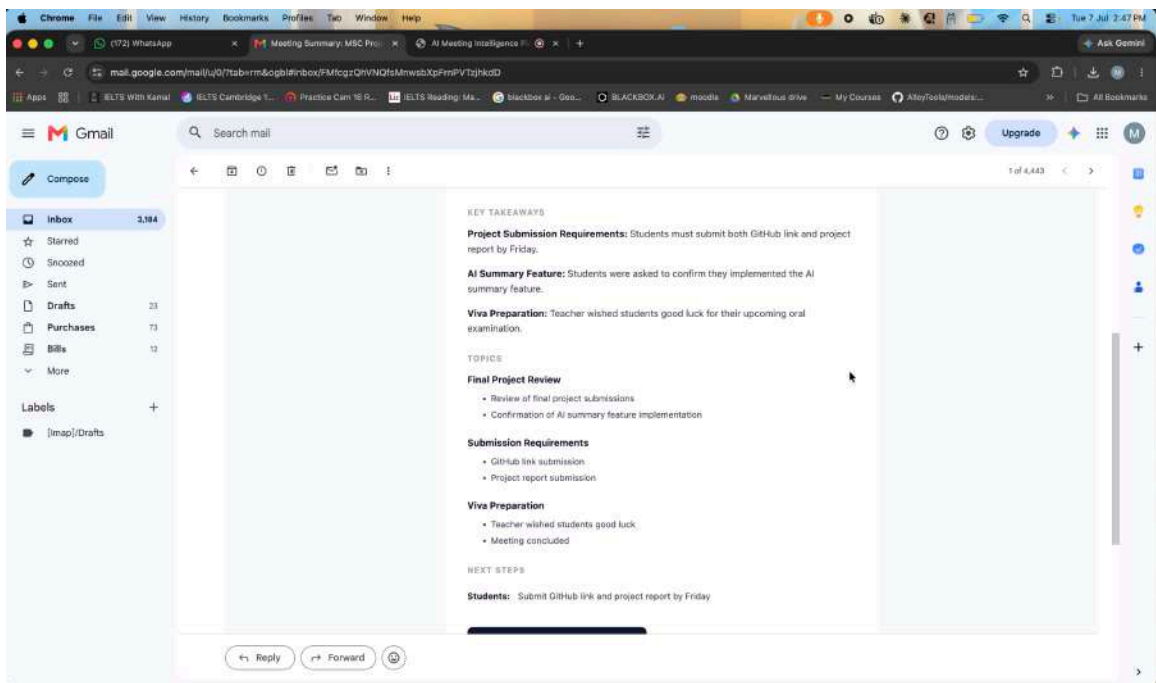


Figure 17.3: Results — Decisions and Action Items

Description

Screenshot showing the Decisions and Action Items section of the email body.

Purpose

To show that the email includes structured, useful information.

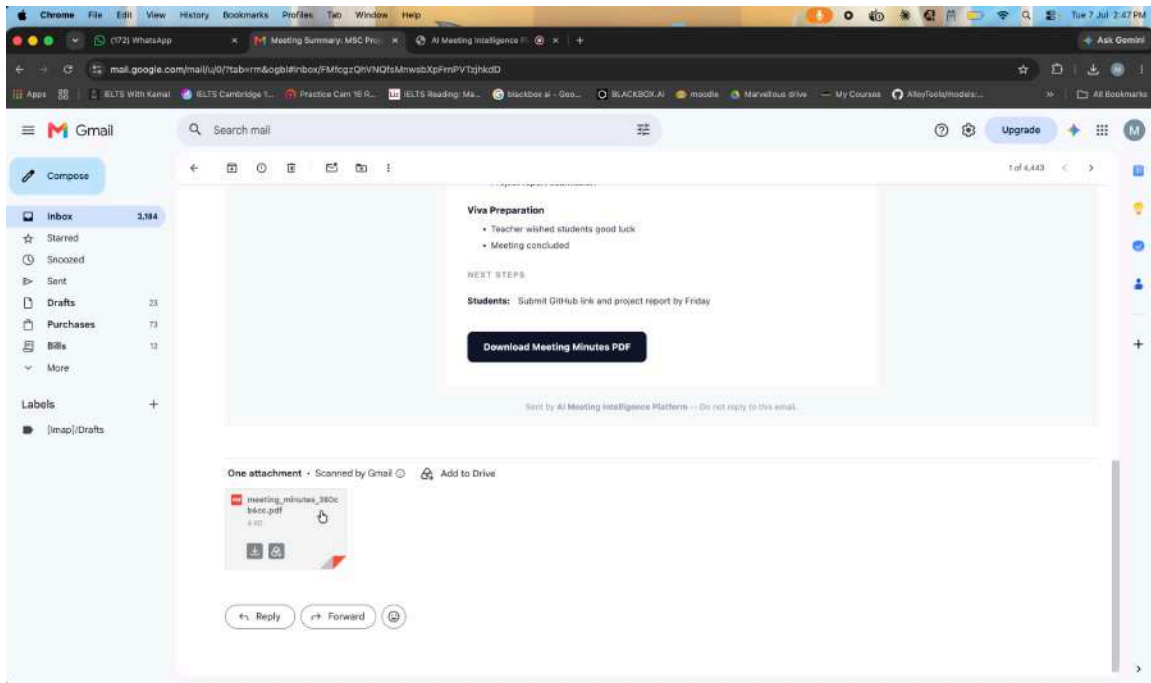


Figure 17.4: Results — PDF Attachment in Email

Description

Screenshot showing the PDF attachment icon in the Gmail email.

Purpose

To confirm that the PDF was correctly attached to the email.

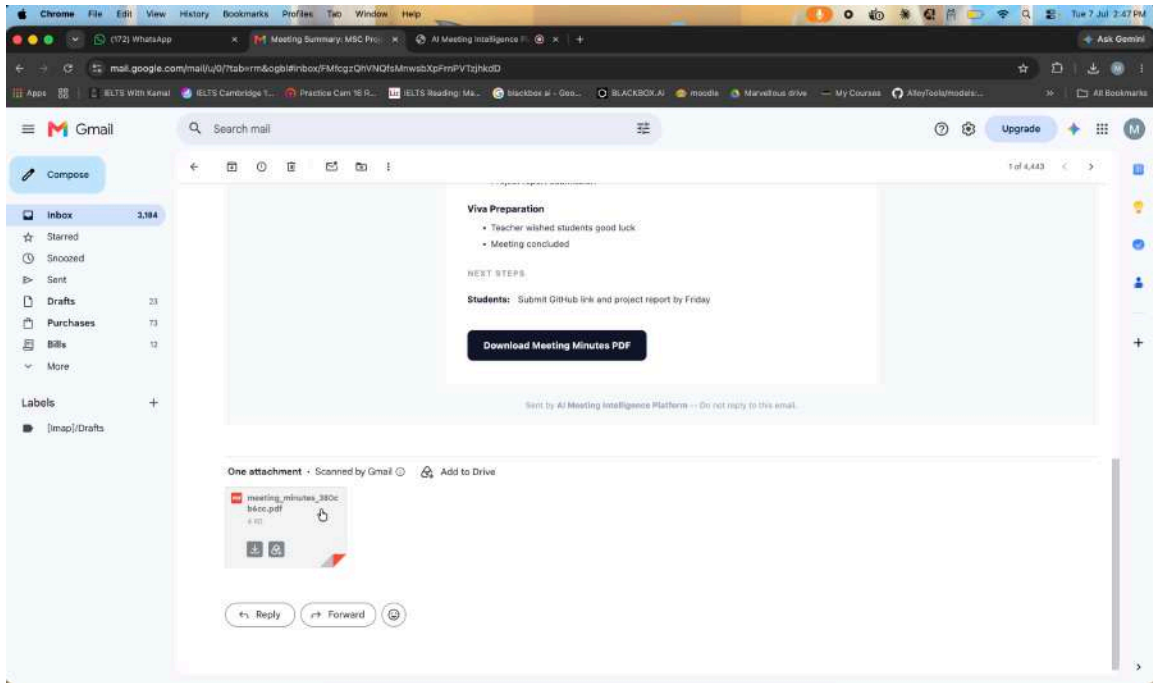


Figure 17.5: Results — Full Email View

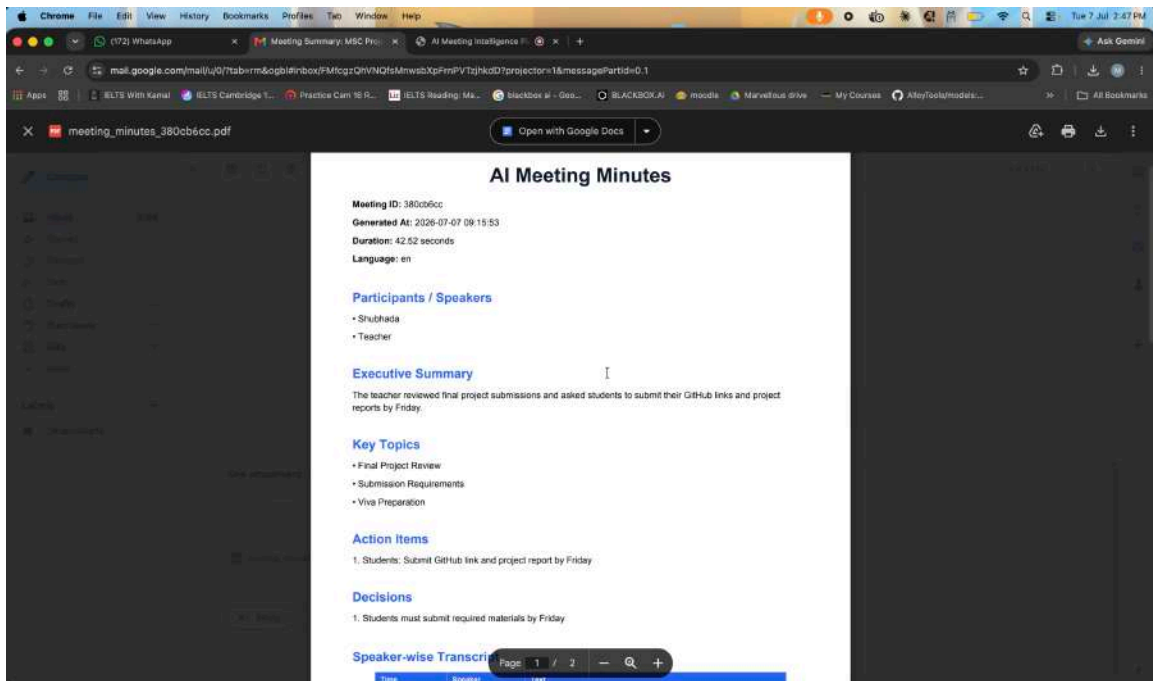


Figure 17.6: Results — PDF Cover Page

Description
and participant list.

Screenshot of the PDF meeting minutes document showing the cover page with meeting title, date,
Purpose
To show the professional appearance of the generated PDF

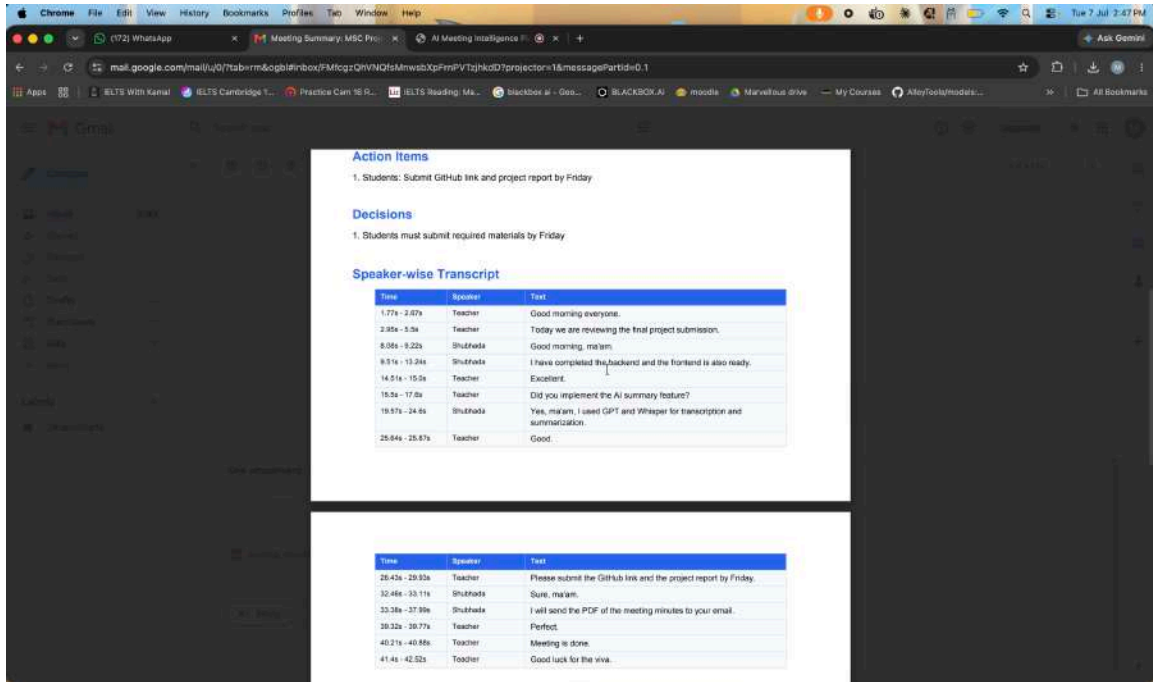


Figure 17.7: Results — PDF Transcript Section

Description

dialogue.

Screenshot of the transcript section inside the PDF, showing speaker labels and timestamped

Purpose

To show the transcript is correctly formatted inside the PDF.

CHAPTER 18: DISCUSSION

18.1 What Worked Well

The transcription fallback pipeline worked exactly as designed. In tests where the internet connection was disabled, the system fell back to faster-whisper and still produced a usable transcript. This proves that the system is genuinely offline-capable, which is the key design goal.

The Docker packaging made deployment straightforward. Setting up the system on a new laptop took less than 15 minutes. All services started automatically and the landing page was accessible immediately.

The RAG-based Q&A system gave accurate, relevant answers for questions about specific parts of the meeting. It correctly cited the relevant speaker and content, and did not hallucinate information that was not in the transcript.

18.2 Challenges Faced

The biggest challenge was speaker diarisation. When two speakers had similar voices or spoke at the same time, the system sometimes misidentified who was speaking. This is a known limitation of current speaker diarisation technology and was partly improved by using the AssemblyAI engine which has better speaker models.

WhisperX requires a GPU to run at a reasonable speed. On a CPU, WhisperX took over 20 minutes to transcribe a 10-minute meeting. This is why faster-whisper was added as a CPU-optimised alternative that processes the same audio in about 4-5 minutes.

Configuring the email SMTP settings correctly required technical knowledge of app passwords (for Gmail) and SMTP server settings. This could be a barrier for non-technical users.

18.3 Comparison with Cloud Solutions

Compared to Zoom AI or Otter.ai, this system has two clear advantages: it is completely free and completely private. The trade-off is that it requires a computer to run as the server and some initial setup. For organisations that handle sensitive data, this trade-off is well worth making.

CHAPTER 19: CONCLUSION

This project successfully built a complete AI-powered meeting platform that works entirely on a local network. All the core features — video meetings, audio recording, speech-to-text transcription, AI summarisation, speaker analytics, Q&A, PDF generation, and email delivery — are working correctly and were tested in a real meeting scenario.

The most important achievement is the four-stage transcription pipeline that ensures a transcript is always produced, even when internet services are unavailable. This makes the system unique among available meeting tools.

The entire system is packaged with Docker and can be started with a single command, making it accessible to non-technical users. It uses only free and open-source software, so there are no ongoing costs to run it.

This project shows that it is possible to build an AI assistant for meetings that is just as capable as expensive commercial products, while maintaining complete privacy and running at zero cost. It could be used by educational institutions, healthcare providers, government offices, or any organisation that values privacy in their meetings.

CHAPTER 20: FUTURE WORK

While the current system achieves all its objectives, there are several enhancements that could be added in future work:

- **Real-time transcription:** Currently transcription happens after the meeting ends. Future work could show a live transcript during the meeting by streaming audio to the transcription engine in small chunks.
- **Mobile app:** A dedicated Android or iOS app would make it easier to join meetings on mobile devices with better camera and microphone control.
- **Multi-language support:** While Sarvam AI handles some Indian languages, improving language detection and adding dedicated models for Hindi, Marathi, and other Indian languages would make the system more useful in Indian educational and government settings.
- **Calendar integration:** Adding a Google Calendar or Outlook integration would allow users to schedule meetings from their existing calendar app.
- **Persistent cloud storage option:** An optional, encrypted cloud backup for recordings and transcripts would allow users who want cloud storage to enable it while keeping it private.
- **Better speaker diarisation:** Training a custom speaker diarisation model on Indian English voices would significantly improve accuracy for typical Indian meeting participants.
- **Admin dashboard:** A web-based admin panel to manage meetings, users, and server resources would make the system easier to operate at an organisational level.

REFERENCES

- [1] LiveKit Inc. (2024). LiveKit Open Source WebRTC Infrastructure. <https://livekit.io>
- [2] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. OpenAI. <https://arxiv.org/abs/2212.04356>
- [3] Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-Accurate Speech Transcription of Long-Form Audio. INTERSPEECH 2023.
- [4] FastAPI Documentation. (2024). FastAPI — modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>
- [5] AssemblyAI. (2024). AssemblyAI Speech Recognition API Documentation. <https://www.assemblyai.com/docs>
- [6] Sarvam AI. (2024). Sarvam AI API — Speech-to-Text for Indian Languages. <https://www.sarvam.ai>
- [7] Docker Inc. (2024). Docker Documentation — Containerization Platform. <https://docs.docker.com>
- [8] Meta AI. (2024). React — A JavaScript library for building user interfaces. <https://react.dev>
- [9] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401.
- [10] OpenAI. (2024). GPT-4 Technical Report. <https://openai.com/research/gpt-4>
- [11] SQLite Consortium. (2024). SQLite Documentation. <https://www.sqlite.org/docs.html>
- [12] ReportLab Inc. (2024). ReportLab PDF Library for Python. <https://www.reportlab.com/docs/>
- [13] RFC 5321 — Simple Mail Transfer Protocol (SMTP). Internet Engineering Task Force (IETF). <https://tools.ietf.org/html/rfc5321>
- [14] Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, 24(6), 381–395.
- [15] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

APPENDIX A: ENVIRONMENT VARIABLES

The system is configured using a .env file placed in the project root directory. Copy .env.example to .env and fill in your values before running docker-compose up.

Variable	Description	Example Value
LIVEKIT_URL	WebSocket URL of the LiveKit server	ws://localhost:7880
LIVEKIT_API_KEY	LiveKit API key for authentication	devkey
LIVEKIT_API_SECRET	LiveKit API secret	devsecret
SARVAM_API_KEY	Sarvam AI API key (optional)	sarvam_xxxx
ASSEMBLYAI_API_KEY	AssemblyAI API key (optional)	asm_xxxx
OPENAI_API_KEY	OpenAI API key for LLM (optional)	sk-xxxx
SMTP_HOST	SMTP server hostname	smtp.gmail.com
SMTP_PORT	SMTP server port	587
SMTP_USER	Email address to send from	you@gmail.com
SMTP_PASS	App password for SMTP	your_app_password
DATABASE_URL	SQLite database file path	sqlite:///./meetings.db

APPENDIX B: API QUICK REFERENCE

All API endpoints accept and return JSON. The base URL for local deployment is `http://localhost:8000`.

Endpoint	Method	Request Body	Response
<code>/meetings</code>	POST	<code>{ "title": "...", "description": "..." }</code>	<code>{ "id": "...", "join_url": "..." }</code>
<code>/meetings/{id}/start-recording</code>	POST	None	<code>{ "status": "recording" }</code>
<code>/meetings/{id}/stop-recording</code>	POST	None	<code>{ "status": "processing" }</code>
<code>/meetings/{id}/transcript</code>	GET	None	<code>{ "turns": [...], "speakers": [...] }</code>
<code>/meetings/{id}/summary</code>	GET	None	<code>{ "summary": "...", "decisions": [...] }</code>
<code>/meetings/{id}/ask</code>	POST	<code>{ "question": "..." }</code>	<code>{ "answer": "..." }</code>
<code>/meetings/{id}/send-email</code>	POST	<code>{ "to": "email@..." }</code>	<code>{ "sent": true }</code>